

局和逻辑布局。

作为例子考虑图 4-47 中的桥接 LAN，其中 9 台机器属于 G（灰色）VLAN，5 台机器属于 W（白色）VLAN。位于灰色 VLAN 中的机器可以分散在两个交换机上，包括通过一个集线器连到交换机上的两台机器。

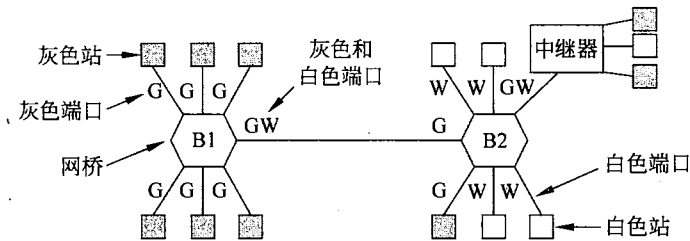


图 4-47 一个桥接 LAN 上的两个 VLAN，分别是灰色的和白色的

为了使 VLAN 正常地运行，网桥必须建立配置表。这些配置表指明了通过哪些端口可以访问到哪些 VLAN。当一帧到来时，比如说来自灰色 VLAN，那么这帧必须被转发到所有标记为 G 的端口。这一条规则对于网桥不知道目的地位置的普通流量（即单播）以及组播和广播流量都适用。注意，一个端口可以标记为多种 VLAN 颜色。

作为一个例子，假设插入在图 4-47 中网桥 B1 的一个灰色站给一个以前没有观察到的站发送一帧。网桥 B1 将接收此帧，发现它来自于灰色 VLAN 的一台机器，因此它在所有标有 G 的端口上泛洪帧（除了入境端口）。该帧将被发送到连在 B1 上的其他五个站，以及通过链路再发送到与之相连的 B2；在网桥 B2 上，帧同样被转发给标有 G 的所有端口；这样帧被进一步发到灰色站和集线器（集线器再将帧转交给它的所有站）。集线器有两个标签，因为它连接着分属于两个 VLAN 的机器。帧不会被发送到标记不是 G 的其他端口，因为网桥知道通过这些端口（非 G）到不了灰色 VLAN 上的机器。

在我们的例子中，帧从网桥 B1 发送到网桥 B2，因为有灰色 VLAN 上的机器与 B2 连接。再来考查白色 VLAN，我们可以看到网桥 B2 连到网桥 B1 的端口没有被标记 W，这意味着在白色 VLAN 上的帧将不会从网桥 B2 转发到网桥 B1。这种行为是正确的，因为没有白色 VLAN 上的机器被连接到 B1。

IEEE 802.1Q 标准

为了实现这个方案，网桥需要知道入境帧属于哪个 VLAN。如果没有这个信息，例如图 4-47 中，当网桥 B2 从网桥 B1 获取一个帧，它不知道该把帧转发到灰色还是白色 VLAN 上。如果我们在设计一个新型局域网时，要做到这点非常容易，只需在帧头添加一个 VLAN 字段。但对以太网该如何处理呢，毕竟它主宰着 LAN，并且没有留下任何空闲字段可用作 VLAN 标识符。

1995 年，IEEE 802 委员会终于将这个问题提到议事日程上。经过多次讨论之后，非常不可思议的是以太网的帧头被修改了。新的格式于 1998 年发表在 IEEE 标准 802.1Q 中。新格式包含了一个 VLAN 标签（tag）；稍后我们将会讨论该标签。毫不奇怪，改变一些事情，如同建立以太网头，并不是一件微不足道的事情。很快可以想到随之出现的一些问题包括：

- (1) 我们需要抛弃现有成千上百万的以太网卡吗？

(2) 如果不抛弃这些网卡, 谁来生成新的字段?

(3) 对于那些已经达到最大长度的帧该怎么办?

当然, 802 委员会知道这些问题 (只是太痛苦了), 而且也必须拿出可行的解决方案, 这是它的职责。

解决问题的关键是要认识到实际使用 VLAN 字段的只是网桥和交换机, 用户机器不需要知道它们。因此, 在图 4-47 中, 重要的是它们出现在连接网桥的线路上, 而不是连到最终站的线路上。另外, 为了使用 VLAN, 网桥必须能感知 VLAN。这一事实使得设计方案具备了实现的可行性。至于说是否要扔掉所有的现有以太网卡, 答案是否定的。请记住, 802.3 委员会甚至无法让人们把类型字段转变成长度字段。你能想象当他们宣布所有现有的以太网卡都必须扔掉时公众的反应。然而, 新的以太网卡与 802.1Q 兼容, 并且能正确地填写 VLAN 字段。

因为存在一些计算机 (和交换机) 无法感知 VLAN, 因此第一个 VLAN 感知的网桥在帧上添加一个 VLAN 字段, 路径上的最后一个网桥把添加的 VLAN 字段删除。混合拓扑的一个例子如图 4-48 所示。在图中, VLAN 感知的计算机直接生成标记帧 (即 802.1Q), 后续交换时要用到这些标记。阴影符号是 VLAN 感知的, 空白符号是不能感知 VLAN 的。

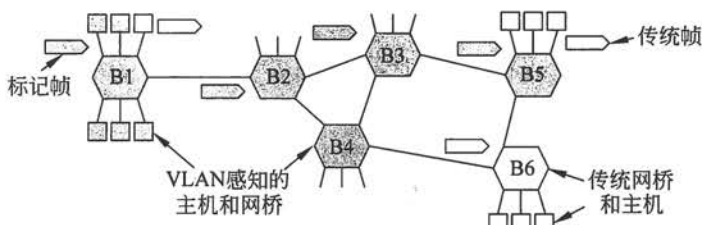


图 4-48 只有部分 VLAN 感知的桥接 LAN
阴影符号是 VLAN 感知的空白符号不能感知 VLAN

有了 802.1Q, 帧被染上颜色, 具体色彩取决于接收它们的端口。对于这种工作方法, 一个端口上的所有机器必须属于同一个 VLAN, 这降低了灵活性。例如, 在图 4-47 中, 这个属性对于把单个计算机连到网桥的所有端口来说是成立的, 但对于集线器连到网桥 B2 的端口来说就不成立了。

此外, 网桥可以使用更高层协议来选择颜色。这样, 到达一个端口的帧或许被放在不同的 VLAN, 要根据其是否携带 IP 数据包或者 PPP 帧而定。

其他方法也是可能的, 但它们没有得到 802.1Q 的支持。举一个例子, 可以用 MAC 地址来选择 VLAN 的颜色, 这或许对来自附近 802.11 局域网的帧有用, 当笔记本电脑在移动时通过不同的端口来发送帧。此时, 把 MAC 地址映射到一个固定的 VLAN, 而不管它从哪个端口进入局域网。

至于帧的长度超过 1518 字节这个问题, 802.1Q 提出把帧长的限制提高到 1522 字节。幸运的是, 只有 VLAN 感知的计算机和交换机必须支持这些长帧。

现在来看 802.1Q 帧的格式。如图 4-49 所示。唯一的变化是加入了一对 2 字节字段。第一个 2 字节是 VLAN 协议标识符 (VLAN protocol ID), 它的值总是 0x8100。由于这个数值大于 1500, 因此, 所有的以太网卡都会将它解释成类型 (type), 而不是长度 (length)。对于这样的帧传统网卡做什么处理都没有意义, 因为这样的帧是不会被发送给传

统网卡的。

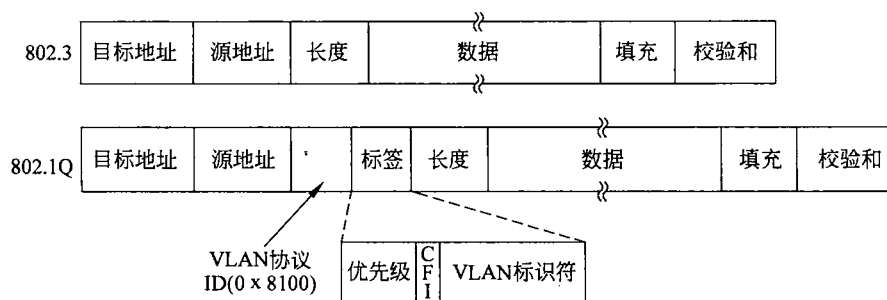


图 4-49 802.3 (传统的) 和 802.1Q 以太网帧格式

第二个 2 字节包含三个子字段。最主要的是 VLAN 标识符 (VLAN identifier) 字段，它占用低 12 位。这正是整个事情的关键所在——帧属于哪种颜色的 VLAN。3 比特的优先级 (Priority) 字段与 VLAN 根本没有关系；但是，由于修改以太网头是 10 年才会发生一次的事件，而且这次事件花了三年时间，涉及上百人，因此为什么在做这件事情的时候不引入一些好的特性呢？这个字段使得交换设备有可能从时间敏感流量中进一步区分出硬实时流量与软实时流量，以及在以太网上提供更好的服务质量。以太网上的话音通信就需要服务质量保证（不过，公平地来讲，IP 有个类似的字段已经存在了四分之一世纪，但是几乎没有人使用它）。

最后一个字段，规范格式指示器 (CFI, Canonical Format Indicator) 应该称为企业自我指示器 (CEI, Corporate Ego Indicator)。它最初的意图是用来指明 MAC 地址中比特的次序 (little-endian 还是 big-endian)，但是，在其他争论中这种用法渐渐被遗忘了。现在，这个比特用来指明有效载荷中是否包含一个冻干了的 802.5 帧。它希望通过以太网可以使得两个相隔遥远的 802.5 局域网用户进行通信，尽管消息需要以太网来承载。当然，这个设想同样跟 VLAN 一点关系也没有。但是，标准委员会的政治学与常规的政治学也没有什么不同：如果你赞成我提出的比特，那么我也会给你的比特投票。

正如我们上面所提到的，当一个标记帧到达一个 VLAN 感知交换机，该交换机利用 VLAN 标识符作为索引，在一张表中查找帧要去哪些端口。但是，这张表从何而来？如果手工来构造，那么我们又回到了问题的起点：手工配置网桥。透明网桥的美丽之处就在于它即插即用，不要求任何手工配置。丢失这个特性简直是一个很可怕的耻辱。幸运的是，VLAN 感知网桥也可以根据它们观察到的标记自动进行配置。如果一个标记为 VLAN4 的帧来自端口 3，那么，很显然端口 3 上有台机器属于 VLAN4。802.1Q 标准说明了如何动态地建立这张表，其中主要引用了在 802.1D 标准中的恰当部分。

在结束 VLAN 路由主题之前，有一种现象值得最后提一下。Internet 和以太网领域中的许多人极其推崇无连接的网络技术，而极力反对任何在数据链路层或者网络层上建立连接的做法。然而，令人惊奇的是，VLAN 实际上引入了一种类似于连接的机制。为了正确使用 VLAN，每个帧携带一个新的特殊标识符，该标识符被当作索引用来查询交换机内部的一张表，从中找出这一帧要去的目标端口。这个过程与面向连接网络惊人的一致。在无连接的网路中，真正被用来路由的是目标地址，而不是某种形式的连接标识符。我们将在第 5 章更详细地讨论这种匍匐式联结。

4.9 本章总结

有些网络只有一个信道可用于全部的通信。在这些网络中，关键的设计问题在于如何在希望使用信道的竞争站之间分配信道。当站的数目较小并且固定，而且流量呈连续型时，FDM 和 TDM 是简单而有效的分配方案。这两种模式被广泛使用在这种情况下，例如，电话中继线带宽的划分。然而，当站的数目较大并且可变，或者流量呈现相当的突发时——正是计算机网络的一般情况，FDM 和 TDM 就是糟糕的选择。

许多动态信道分配算法被设计了出来。ALOHA 协议，包括分槽的与不分槽的，被用在许多实际系统的衍生物中，例如线缆调制解调器和 RFID。当可以侦听信道状态后，作为一种改进，站在其他站传输时可避免启动自己的传输。这种技术，即载波侦听，导致了局域网和城域网的 CSMA 各种协议。它是典型以太网和 802.11 网络的基础。

众所周知有一类协议能完全消除竞争，或者至少能大大减少了竞争。位图协议、拓扑结构（比如环）以及二进制倒计时协议完全消除了竞争。树遍历协议能减少竞争，具体做法是动态划分成两个大小不同的相邻组；并且只允许同一个组内的站竞争；最理想的组选择是当它允许发送时只有一个站要发送。

无线局域网提出了新的问题，就是很难侦听到传输冲突，而且站所覆盖的区域可能有所不同。在主宰无线局域网的 IEEE 802.11 中，站使用 CSMA/CA，通过留有很小的时间间隔来避免冲突，从而减轻第一个问题。站还可以使用 RTS/CTS 协议来对抗由于第二个问题引起的隐藏终端。IEEE 802.11 通常被用于把笔记本电脑和其他设备连接到无线接入点，但它也可以用在设备之间的自组织联网。任何一种物理层都可以被使用，包括有或没有多个天线的多信道频分复用，以及扩频技术。

跟 802.11 一样，RFID 读写器和标签使用随机访问协议来交换标识符。其他无线个域网和无线城域网有不同的设计。蓝牙系统可连接耳机和许多不同类型的外设到计算机而无须任何线缆。IEEE 802.16 为固定和移动计算机提供了广域的无线 Internet 数据服务。这两类网络都使用了一个集中并面向连接的设计，其中蓝牙主节点和 WiMAX 基站决定每个站何时可以发送或接收数据。对于 802.16 而言，这种设计可为诸如电话那样的实时流量和像 Web 浏览这样的交互式流量提供服务质量保障。至于蓝牙，复杂性放置在主节点上，从而导致从节点非常的价廉物美。

以太网是有线局域网的主要形式。经典以太网使用 CSMA/CD 来分配黄色花园软管内电缆的信道，这个软管在机器之间蜿蜒穿梭。随着速度由原来 10 Mbps 提升到 10 Gbps 并继续攀升，体系结构已经发生改变。现在，诸如双绞线那样的点到点链路连接着集线器和交换机。有了现代交换机和全双工链路，链接上已经不存在竞争，交换机可以在不同的端口之间并行地转发帧。

若要把局域网布满整个建筑物，就需要一种互连局域网的方法。即插即用的网桥就能用于此目的。搭建网桥时需要用到后向学习算法和生成树算法。由于此功能已被纳入现代交换机，因此术语“网桥”和“交换机”可以互换着使用。为了帮助桥接局域网的管理，VLAN 技术使得物理拓扑结构可划分为不同的逻辑拓扑结构。VLAN 标准，即 IEEE 802.1Q

协议，引入了一种新的以太网帧格式。

习 题

1. 在这个练习中，请使用本章中的一个公式，但在计算之前请先说明这个公式。帧随机到达一个 100 Mbps 信道，并等待传输。如果帧到达时信道正忙，那么它必须排队等待。帧的长度呈指数分布，均值为 10 000 位/帧。对于下列每一种帧到达率，试问平均一帧的延迟是多少（包括排队时间和传输时间）？
 - (a) 90 帧/秒。
 - (b) 900 帧/秒。
 - (c) 9000 帧/秒。
2. N 个站共享一个 56 kbps 的纯 ALOHA 信道。每个站平均每 100 秒输出一个 1000 位长的帧，即使前面的帧还没有被发送出去（比如，站可以将出境帧缓存起来）。试问 N 的最大值是多少？
3. 考虑在低负载情况下纯 ALOHA 和分槽 ALOHA 的延迟。试问哪个延迟更小？请说明你的答案。
4. 一大群 ALOHA 用户每秒钟产生 50 个请求，包括原始的请求和重传的请求。时间槽单位为 40 毫秒。
 - (a) 试问：第一次发送成功的机会是多少？
 - (b) 试问：恰好 k 次冲突之后成功的概率是多少？
 - (c) 试问：所需传输次数的期望值是多少？
5. 在一个有无限用户的分槽 ALOHA 系统中，一个站在冲突之后到重传之间的平均等待时间槽数目为 4。请画出该系统的延迟与吞吐量之间的关系图。
6. 试问在下列两种情况下 CSMA/CD 的竞争时间槽长度是多少？
 - (a) 一个 2 千米长的双导电缆（twin-lead cable）（信号的传播速度是信号在真空中传播速度的 82%）？
 - (b) 40 千米长的多模光纤（信号的传播速度是信号在真空中传播速度的 65%）？
7. 在一个使用基本位图协议的局域网中，最坏的情况下一个站（比如 s）要等多久才可以传输它的帧？
8. 在二进制倒数计数协议中，试问为什么一个编号较低的站有可能得不到发送数据包的机会。
9. 编号为 1~16 的 16 个站使用自适应树遍历协议来竞争一个共享信道。如果所有站的地址预先设定，并且突然一次就绪，试问解决竞争需要多少个比特槽？
10. 考虑 5 个无线站：A、B、C、D 和 E。站 A 可与所有其他站通信。B 可以与 A、C 和 E 通信。C 可以与 A、B 和 D 通信。D 可以与 A、C 和 E 通信。E 可以和 A、D 和 B 通信。
 - (a) 当 A 给 B 发送时，试问可能进行的其他通信是什么？
 - (b) 当 B 给 A 发送时，试问可能进行的其他通信是什么？
 - (c) 当 B 给 C 发送时，试问可能进行的其他通信是什么？

11. 6个站的编号从A到F，它们使用MACA协议进行通信。试问有可能同时发生两个传输操作吗？请说明你的答案。
12. 一个七层办公楼的每一层有15个相邻的办公室。每个办公室的前面墙上包含一个终端插口。所以，在垂直面上，这些插口构成了一个矩形网格，在水平方向和垂直方向上插口之间均有4米远的距离。假定在任何一对插口之间，无论是水平的、垂直的，或是对角的，都可以直接拉一根线缆，试问若使用下面的配置需要多少米线缆才能将所有的插口连接起来：
 - (a) 正中间放置一台路由器的星型结构。
 - (b) 经典802.3 LAN。
13. 试问经典10 Mbps以太网的波特率是多少？
14. 假设经典以太网使用曼彻斯特编码，请画出比特流0001110101的编码输出。
15. 一个1千米长、10 Mbps的CSMA/CD LAN（不是802.3），其传播速度为200米/微秒。这个系统不允许使用中继器。数据帧的长度是256位，其中包括32位的头、校验和以及其他开销。在一次成功传输后的第一个比特槽被预留给接收方，以便它抓住信道发送32位的确认帧。假定没有冲突，试问除去开销之后的有效数据率是多少？
16. 两个CSMA/CD站都企图传送大文件（多个帧）。每发出一帧，它们就使用二进制指数后退算法竞争信道。试问在第k轮结束竞争的概率是多少？每个竞争周期的平均次数是多少？
17. 一个通过以太网传送的IP数据包长60字节，其中包括所有的头。如果没有使用LLC，试问需要往以太网帧中填补字节吗？如果需要，试问需要填补多少个字节？
18. 以太网帧必须至少64字节长，才能确保当电缆另一端发生冲突时，发送方仍处于发送过程中。快速以太网也有同样的64字节最小帧长度限制，但是它可以快10倍的速度发送数据。试问它如何有可能维持同样的最小帧长度限制？
19. 有些书将以太网帧的最大长度说成是1522字节而不是1500字节。它们错了吗？请说明你的回答。
20. 试问千兆以太网每秒钟能够处理多少个帧？请仔细想一想，并考虑所有相关情形。提示：请考虑千兆以太网的实质。
21. 请说出两个网络，它们允许将多个连续的帧背靠背地打包在一起。试问为什么这个特性值得专门提出来？
22. 在图4-27中有4个站A、B、C和D。试问你认为后两个站中哪一个最接近A？为什么？
23. 试举例说明802.11协议中的RTS/CTS与MACA协议有啥点不同。
24. 一个无线局域网内有一个AP和10个客户站。4个站的数据速率为6 Mbps，另外4个站有18 Mbps的数据速率，最后两个站有54 Mbps的数据速率。试问当全部10个站一起发送数据，并且下列条件成立时，每个站能获得的数据速率是多少？
 - (a) 没有用TXOP。
 - (b) 采用了TXOP。
25. 假设一个11 Mbps的802.11b LAN正在无线信道上传送一批连续的64字节帧，比特错误率为 10^{-7} 。试问平均每秒钟将有多少帧被损坏？
26. 一个802.16网络有一个20 MHz宽的信道。试问可以多大的bps给固定用户站发送？

27. 为什么有些网络用纠错码而不用检错和重传机制？请给出两个理由。
28. 请分别列出 WiMAX 与 802.11 类似的两种方法和它与 802.11 不同的两种方法。
29. 从图 4-34 中，我们可以看到一个蓝牙设备可同时位于两个微网中。试问有理由说明为什么一个设备不可能同时是这两个微网中的主节点？
30. 试问在基本速率下，一个 3 槽蓝牙帧的数据字段最大长度是多少？请解释你的答案。
31. 图 4-24 显示了几种物理层协议。试问这些协议中哪个最接近蓝牙物理层协议？它们之间的最大差异是什么？
32. 在 4.6.6 节中提到，基本速率下一个 1 槽帧重复编码后的效率约为 13%，试问基本速率下一个 5 槽帧重复编码后的效率是多少？
33. 在 802.11 的跳频扩展频谱变种中，信标帧包含了停留时间。试问你认为蓝牙中类似的信标帧也包含了停留时间吗？请讨论你的答案。
34. 假设有 10 个 RFID 标签围绕在 RFID 读写器的周围。试问最好的 Q 值是多少？在给定的槽内一个标签无冲突响应的可能性有多大？
35. 请列出 RFID 系统的一些安全隐患。
36. 一个专门为快速以太网设计的交换机有一个传输速率为 10 Gbps 的背板。试问在最差情况下它可以多大的帧/秒来处理帧？
37. 请简单描述存储-转发型交换机和直通型交换机之间的区别。
38. 考虑图 4-41 (b) 用网桥 B1 和 B2 连接的扩展局域网。假设两个网桥的哈希表是空的。对于下面的数据传输序列，请列出转发数据包所用的全部端口：
 - (a) A 发送一个数据包给 C。
 - (b) E 发送一个数据包给 F。
 - (c) F 发送一个数据包给 E。
 - (d) G 发送一个数据包给 E。
 - (e) D 发送一个数据包给 A。
 - (f) B 发送一个数据包给 F。
39. 从损坏帧的角度来看，存储-转发型交换机比直通型交换机更有优势。请说明这种优势是什么。
40. 本章 4.8.3 节中提到，一些网桥甚至可能不会出现在生成树种。请描绘一个场景，其中一个网桥可能无法出现在生成树中。
41. 为了使得 VLAN 正常工作，在网桥内部需要有相应的配置表。如果图 4-47 中的 VLAN 使用集线器而不是交换机，情况会怎么样呢？集线器也需要配置表吗？为什么需要，或者为什么不需要？
42. 在图 4-48 中，右侧传统终端域中的交换机是一个 VLAN 感知交换机。试问在那里有可能使用传统的交换机吗？如果可能，试问它如何工作？如果不可能，请问为什么？
43. 请编写一个程序来模拟以太网上 CSMA/CD 协议的行为：当一帧正在被发送时，有 N 个站都准备要发送。你的程序应该报告每一个站成功开始发送帧的时间。假设每个时间槽（51.2 微秒）时钟滴答一次，并且冲突检测和发送干扰序列只需要一个时间槽。所有帧都具有最大允许的长度。

网络层关注的是如何将源端数据包一路送到接收方。为了将数据包送到接收方，可能沿途要经过许多跳(hop)中间路由器。这种功能显然与数据链路层的功能不同，数据链路层的目标没那么宏伟，只是将帧从线路一边传送到另一边。因此，网络层是处理端到端数据传输的最底层。

为了实现这个目标，网络层必须知道网络拓扑结构(即所有路由器和链路的集合)，并从中选择出适当的路径，即使是大型网络也要选出一条好路径。同时，网络层还必须仔细选择路由器，避免某些通信线路和路由器负载过重，而其他线路和路由器空闲。最后，当源端和接收方位于不同网络时，还会出现新的问题，这些问题都需要由网络层来解决。在本章，我们将讨论所有这些问题，并主要结合 Internet 及其网络层协议(IP)来深入探讨。

5.1 网络层的设计问题

在本节，我们将简要描述网络层设计人员必须重视的一些问题，其中包括为传输层提供的服务以及网络的内部设计。

5.1.1 存储转发数据包交换

在介绍网络层细节之前，有必要再次说明网络层协议运行的上下文。图 5-1 大致勾画出了这样的环境。网络中最主要的组件是网络服务提供商(ISP)的设备(通过传输线路连接的路由器)和客户端设备，在图中 ISP 的设备位于阴影椭圆内，而客户设备位于椭圆之外。主机 H1 直接连接到 ISP 的路由器 A，这或许是一台家用计算机，通过 DSL 调制解调器接入；而 H2 则位于一个局域网内，这可能是一个办公室以太网，其上还有台路由器 F，客户拥有这台路由器并负责其运行。路由器 F 通过一条租用线路连接到 ISP 的设备上。然而，为了本章的目的，还是把客户端的路由器作为 ISP 网络的一部分来考虑，因为它们运行的算法与 ISP 路由器上运行的算法相同(我们本章关注的就是算法)。

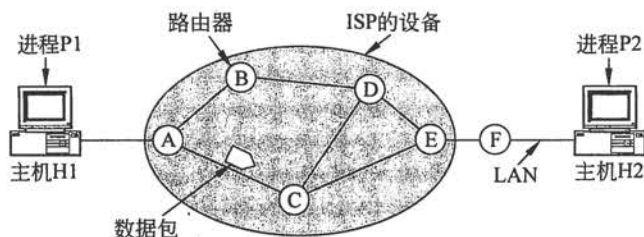


图 5-1 网络层协议的环境

这种网络配置的使用方式如下所述。如果一台主机要发送一个数据包，它就将数据包

传输给最近的路由器，路由器可能在它自己的 LAN 上，也可能在一条通向 ISP 的点到点链路上。在该数据包到达路由器，并且路由器的链路层完成了对它校验和的验证之后，它先被存储在路由器上；然后沿着路径被转发到下一个路由器，直至到达目标主机，这里就是数据包的目的地。这种机制即为存储-转发数据包交换，正如在前面几章已经看到的那样。

5.1.2 提供给传输层的服务

网络层通过网络层/传输层接口向传输层提供服务。一个重要的问题是明确网络层向传输层提供什么类型的服务。在设计网络层服务时，一定要牢记下面这些目标：

- (1) 向上提供的服务应该独立于路由器技术。
- (2) 应该向传输层屏蔽路由器的数量、类型和拓扑关系。
- (3) 传输层可用的网络地址应该有一个统一编址方案，甚至可以跨越 LAN 和 WAN。

给定这些目标后，网络层设计者有很大的自由度来编写提供给传输层的详细服务规范。这种自由度通常演变为两个竞争派别之间的激烈争斗。最终讨论的焦点集中在网络层应该提供面向连接的服务还是提供无连接的服务。

一个阵营（以 Internet 社团为代表）认为，路由器的任务仅仅是传送数据包，不用再做别的事情。按照他们的观点（基于 40 年来从一个实际计算机网络获得的经验），不管如何设计网络，从本质上讲它总是不可靠的。因此，主机应该接受这样的事实，自己来完成错误控制（即错误检测和纠正）和流量控制任务。

这种观点很快地导致了这样的结论：网络服务应该是无连接的，只需要原语 SEND PACKET 和 RECEIVE PACKET，以及少量其他的原语就够了。特别是，数据包的排序和流量控制不应该在这里完成，因为主机将会完成这些工作，做两遍同样的工作通常不会带来更多好处。这个推理就是端-端论点的例子，这种设计理念对 Internet 形成有着很大的影响。而且，每个数据包必须携带完整的目标地址，因为每个数据包的运送独立于它前面的那些数据包（如果此前有数据包的话）。

另一大阵营（以电话公司为代表）认为，网络应该提供可靠的、面向连接的服务。他们声称，具有 100 多年成功经验的全球电话系统就是一个极好的范例。按照他们的观点，服务质量是最主要的因素，并且如果在网络中没有连接，要实现服务质量非常困难，特别对于诸如语音和视频这样的实时流量。

即使几十年以后，这场争论仍然十分活跃。早期被广泛使用的数据网络都是面向连接的，比如 20 世纪 70 年代的 X.25 和 80 年代其继任者帧中继（Frame Relay）。然而，自从有了 ARPANET 和早期 Internet，无连接网络层得到了突飞猛进的普及。现在 IP 协议俨然是一个无处不在的成功象征。虽然在 20 世纪 80 年代，它受到一种叫做 ATM 的面向连接技术的威胁，该技术就是为了推翻 IP 而开发的；结果刚好相反，现在 ATM 终于找到了自己的用武之地，而 IP 正在接管电话网络。然而，在幕后，Internet 正朝着面向连接的特性进化，因为服务质量变得越来越重要了。与此相关的两个面向连接技术的例子是多协议标签交换（MPLS, MultiProtocol Label Switching）和 VLAN，我们将在本章描述 MPLS，有关 VLAN 我们在第 4 章中已经介绍过了。目前这两种技术已经被广泛使用。

5.1.3 无连接服务的实现

在考查了网络层所提供的两类服务之后，我们现在来看网络层内部是如何工作的。根据提供的服务类型，可能存在两种不同的组织方式。如果提供的是无连接的服务，那么，所有的数据包都被独立地注入到网络中，并且每个数据包独立路由，不需要提前建立任何设置。在这样的上下文中，数据包通常称为数据报（datagram），它类似于电报（telegram），对应的网络称为数据报网络（datagram network）。如果使用了面向连接的服务，那么，在发送数据包之前，必须首先建立起一条从源路由器到目标路由器之间的路径。这个连接称为虚电路（VC，virtual circuit），它类似于电话系统中建立的物理电路，对应的网络称为虚电路网络（virtual-circuit network）。在本节，我们将讨论数据报网络；在下节，我们再讨论虚电路网络。

现在我们来查看数据报网络是如何工作的。假设图 5-2 中的进程 P1 有一个很长的消息要发送给 P2。它将消息递交给传输层，并指示传输层将消息传送给主机 H2 上的进程 P2。传输层代码运行在 H1 上，通常在操作系统内部。它在消息的前面加上一个传输头，然后将结果交给网络层，这里的网络层可能是操作系统内部的另一个过程。

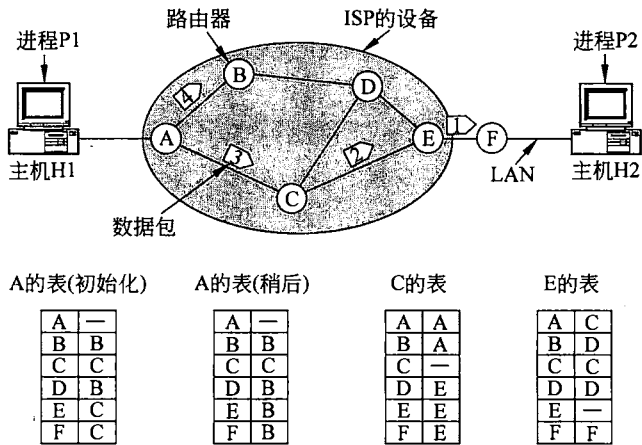


图 5-2 数据报网络中的路由过程

让我们假设在这个例子中消息的长度是最大数据包长度的 4 倍，所以，网络层必须将消息拆分成 4 个数据包：1、2、3 和 4，然后用某种点到点协议（比如 PPP）将这些数据包依次发送给路由器 A。到这里，ISP 将消息的传输任务接管过来。每一台路由器都有一个内部表，它指明了针对每一个可能的目标地址应该将数据包送到哪里去。每个表项由两部分数据组成：目标地址和通往目标地址所使用的出境线路。当然，只能使用直接连接的线路。例如，在图 5-2 中，A 只有两条出境线路——分别通向 B 和 C，所以，每一个入境数据包必须被转发给这两台路由器之一，即使它最终目标地址是其他某一台路由器。A 的初始路由表如图中标示的“初始化”（initially）。

在路由器 A，数据包 1、2 和 3 分别到达入境线路并且经过验证校验和之后，被路由器暂时保存起来。然后，根据 A 上的表，每个数据包被放在一个新帧中，并且被转发到通往

C 的出境链路；之后数据包 1 被转发给 E，进一步又被转发给 F。当它到达 F 时，它被封装在一个帧内通过连有 H2 的 LAN 被发送出去。数据包 2 和 3 遵循同样的路径。

然而，数据包 4 的情形有所不同。当它到达 A 之后，尽管它的目标地址也是指向 F，但它被 A 转发给了路由器 B。出于某种原因，A 决定采用不同于前三个数据包的路径来发送数据包 4。或许它了解到在 ACE 路径上发生了流量拥塞，因而更新了路由表，如图中标示的“稍后”（later）。管理这些路由表并做出路由选择的算法称为路由算法（routing algorithm）。路由算法是我们本章学习的一个主要议题，正如我们将会看到的那样，有几种不同类型的路由算法。

IP 协议（Internet Protocol）是整个 Internet 的基础，它是无连接网络服务的重要范例。每个数据包携带一个目标 IP 地址，路由器使用该地址来单独转发每一个数据包。IPv4 数据包的地址是 32 位，IPv6 数据包的地址是 128 位。我们在本章后面部分将详细描述 IP。

5.1.4 面向连接服务的实现

对于面向连接的服务，我们需要一个虚电路网络。现在我们讨论它是如何工作的。隐藏在虚电路背后的思想是避免为每个要发送的数据包选择一条新路径（像图 5-2 那样）。相反，当建立一个连接时，从源机器到目标机器之间的一条路径就被当作这个连接的一部分确定了下来，并且保存在这些中间路由器的表中。所有需要在这个连接上通过的流量，都使用这条路径，这与电话系统的工作方式完全一致。当连接被释放之后，虚电路也随之消失。在面向连接的服务中，每个数据包包含一个标识符，指明了它属于哪一条虚电路。

作为一个例子，请考虑图 5-3 的情形。在这里，主机 H1 已经建立了一条与主机 H2 之间的连接 1。这条连接被记录在每个路由表中的第一项中。A 路由表的第一行说明如果一个标示了连接标识符 1 的数据包来自于 H1，那么它将被发送到路由器 C，并且赋予连接标识符 1。类似地，C 路由表中的第一项将该数据包路由到 E，也赋予连接标识符 1。

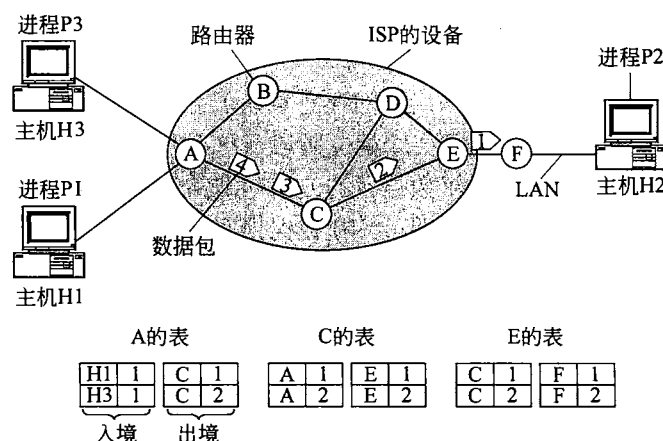


图 5-3 虚电路网络的路由过程

现在我们来考虑如果 H3 也希望与 H2 建立连接则情形会怎么样。H3 选择连接标识符 1（因为是它发起连接，而且这是它唯一的连接），并且告诉网络要建立虚电路。因此路由表增加了第二行。请注意，这里有一个冲突，因为，尽管 A 很容易区分出标识连接 1 的数

据包是来自 H1 还是来自 H3，但是，C 无法区分它们。基于这个原因，A 给第二个连接的出境流量分配一个不同的连接标识符。这种避免冲突的做法说明了为什么路由器需要具备替换出境数据包中连接标识符的能力。

在有些上下文中，这个过程称为**标签交换**（label switching）。一种面向连接的网络服务例子是**多协议标签交换**（MPLS，MultiProtocol Label Switching），它主要被用在 Internet 的 ISP 网络，IP 数据包被一个有 20 位连接标识或标签的 MPLS 头包裹着。MPLS 往往对客户是隐藏的，客户看不到这些标签，ISP 用它来为超大流量建立长期的连接；但是，当服务质量变得很重要而且还需要协助其他 ISP 完成流量管理任务时，MPLS 的作用越来越突出。我们将在本章后面更多地介绍有关 MPLS 的细节。

5.1.5 虚电路与数据报网络的比较

虚电路和数据报都有各自的支持者和反对者。我们现在试图从多个角度对两组论点作个总结。图 5-4 列出了最主要的问题，尽管纯粹主义者总是有可能找到图中每一项的反例。

问题	数据报网络	虚电路网络
电路建立	不需要	需要
寻址	每个包包含全部的源和目标地址	每个包包含简短的VC号
状态信息	路由器不保留连接状态	针对每个连接，每条VC都需要路由器保存其状态
路由方式	每个数据包被单独路由	建立VC时选择路由，所有包都遵循该路由
路由器失效的影响	没影响，除了那些路由器崩溃期间丢失的包	穿过故障路由器的所有VC都将中断
服务质量	困难	容易，如果在预先建立每条VC时有足够的资源可分配
拥塞控制	困难	容易，如果在预先建立每条VC时有足够的资源可分配

图 5-4 数据报网络和虚电路网络的比较

在网络内部，数据报和虚电路网络之间存在着几个方面的权衡。一个权衡在于建立时间和地址解析时间。使用虚电路需要一个建立阶段，这个阶段既花费时间也消耗资源。然而，一旦付出了这个代价，处理一个数据包的方法却非常简单：路由器只要使用电路号作为索引，在表中找到该数据包的去向即可。在数据报网络中，不需要建立电路，但路由器需要执行一个更为复杂的查找过程以便找到目标表项。

与此相关的问题是数据报网络所用的目标地址比虚电路网络所用的电路号要长，因为数据报网络的目标地址具备全局意义。如果数据包相当短，在每个数据包中都包括完整的目标地址可能意味着大量的协议开销，因而造成带宽资源的浪费。

另一个问题是路由器内存所要求的表空间的数量。在数据报子网中，针对每一个可能的目标地址都要求有一个表项，而在虚电路网络中，只要为每一条虚电路提供一个表项即可。然而，这种优势有点虚幻并非绝对，因为虚电路网络在建立连接阶段所用的数据包也需要被路由，并且它们也使用目标地址，如同数据报网络的做法一样。

从保证服务质量以及避免网络拥塞的角度来看，虚电路有一定的优势，因为在建立连

接时，资源可以提前预留（比如缓冲区空间、带宽和 CPU 周期）。一旦数据包开始到来，所需要的带宽和路由器容量都已经准备就绪。而对于数据报网络，避免拥塞更困难些。

对于事务处理系统（比如商场购物时通过电话验证信用卡的有效性），用于建立和清除虚电路所需要的开销有可能会削弱虚电路的优势。如果系统中大部分流量都是这种类型，那么在网络内部使用虚电路就毫无意义。另一方面，公司的两个办公楼之间长期运行诸如 VPN 流量，这种情况下永久性的虚电路或许更加有用（可手工建立虚电路，并且持续使用几个月或者几年）。

虚电路也存在脆弱性问题。如果一台路由器崩溃并且内存中的数据全部丢失，那么即使它一秒钟之后又重新启动，所有从它这里经过的虚电路都将不得不中断。相反，如果一台数据报路由器宕机，只有那些当时尚留在路由器队列中的数据包用户会受到影响（甚至这些用户也不会全部受到影响，因为发送方可能很快重传这些数据包）。一条通信线路的失败对于使用了该线路的虚电路来说是致命的，但如果使用数据报，则这种失败很容易得到弥补。数据报还允许路由器平衡网络流量，因为一个长序列数据包的传输路径可以在序列传输的中途改变。

5.2 路由算法

网络层的主要功能是将数据包从源机器路由到目标机器。在大多数网络中，数据包需要经过多跳（hop）才能到达目的地。唯一一个值得指出的例外是广播网络，但即使在广播网络中，如果源机器和目标机器不在同一个网络段中时，路由仍然是一个问题。选择路由的算法以及这些算法所用的数据结构是网络层设计的最主要内容。

路由算法（routing algorithm）是网络层软件的一部分，它负责确定一个入境数据包应该被发送到哪一条输出线路上。如果网络内部使用了数据报，那么路由器必须针对每一个到达的数据包重新选择路径，因为自上一次选择了路径之后，最佳路径可能已经发生了改变。如果网络内部使用虚电路，那么只有当建立一条新的虚电路时，才需要做路由决策；此后，数据包只要沿着已经建立的路径向前传递即可。后一种情形有时候也称为会话路由（session routing），因为在整个会话过程中（比如 VPN 上的一个终端登录会话），路径必须保持有效。

有的时候对路由和转发这两个功能进行区分是非常有用的，路由即对使用哪一条路径做出决策，而转发则是当一个数据包到达时该采取什么动作。可以把路由器想象成内部有两个进程。其中一个进程在每个数据包到达的时候对它进行处理，它在路由表中查找该数据包所对应的出境线路。这个进程即为转发（forwarding）进程；另一个进程负责生成和更新路由表，这正是路由算法发挥作用的地方。

无论是针对每个数据包独立选择路由，还是仅在建立新连接时选择路径，路由算法必须满足某些特性：正确性、简单性、鲁棒性、稳定性、公平性和有效性。正确性和简单性无须多加解释，但乍一看对鲁棒性的要求则没有那么显而易见了。一旦一个重要网络投入运行，它有可能需要连续运行数年而不能出现波及系统范围的失败。在此期间，将会有各种各样的硬件和软件发生故障。主机、路由器和线路可能会重复失败，网络拓扑结构也可能

会多次发生变化。路由算法应该能够处理拓扑结构和流量方面的各种变化，而且不能要求所有主机都停止所有的工作。可以想象每次路由器崩溃时都需要网络重新启动的严重后果！

稳定性对于路由算法来说也是一个重要目标。有一些路由算法无论运行多长时间从来都没有收敛到一个固定的路径集合。一个稳定的算法能达到平衡，并且保持平衡状态。因此它应该迅速收敛，在路由算法达到平衡之前，通信可能无法正常进行。

公平性和有效性听起来显然是理所应当的——肯定不会有人反对这两种特性。但是，事实证明它们往往是两个相互矛盾的目标。举一个简单的例子来说明这种冲突，请看图 5-5。假设在 A 和 A' 之间、B 和 B' 之间以及 C 和 C' 之间有足够的流量使得水平的链路达到饱和。为了使总流量达到最大，X 和 X' 之间的流量应该完全被切断。不幸的是，X 和 X' 可能看不到这一点。很显然，在全局效率和单个连接的公平性之间必须有一种折中的处理办法。

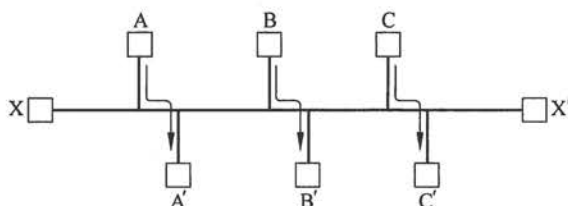


图 5-5 公平性和效率产生冲突的网络

我们在试图找到公平性和有效性之间的权衡办法之前，首先必须确定要优化什么性能指标。使数据包的平均延迟达到最小是有效发送流量的一种很明显的选择，但是使网络的总吞吐量最大化也是一种不错的选择。而且，这两个目标也是相互冲突的，因为运行任何一个接近容量的排队系统意味着排队延迟很长。作为一种折中，许多网络企图最小化一个数据包必须经过的跳数或简单降低其经历的跳数。两种选择趋向于减小延迟，同时减少每个数据包消耗的带宽数量，从而来提高整个网络的吞吐量。

路由算法可以分成两大类：非自适应算法和自适应算法。非自适应算法（nonadaptive algorithm）不会根据当前测量或者估计的流量和拓扑结构，来调整它们的路由决策。相反，从 I 到 J（对所有的 I 和 J）所使用的路由选择是预先在离线情况下计算好，并在网络启动时被下载到路由器中的。这个过程有时候也称为静态路由（static routing）。因为它无法响应故障，所以静态路由对于路由选择已经很清楚的场合非常有用。例如，在图 5-3 中，路由器 F 应该通过路由器 E 把数据包发到网络中，根本不管数据包的最终目的地在哪里。

与此相反，自适应算法（adaptive algorithm）则会改变它们的路由决策以便反映出拓扑结构的变化，通常也会反映出流量的变化情况。这些动态路由（dynamic routing）算法在多个方面有所不同：获取信息的来源不同（例如，来自于本地、相邻路由器，或者所有的路由器）、改变路径的时间不同（比如，每当拓扑发生变化时，或者每隔 Δt 秒随负载变化）以及用于路由优化的度量不同（比如，距离、跳数或者估计的传输时间）。

在下面的章节中，我们将讨论不同的路由算法。算法除了从源端发送数据包到接收方外，还涉及传递模式。有时候路由的目标是把数据包发送到多个地址、全部地址或者一组目标地址中的一个。我们在这里描述的所有路由算法都基于拓扑结构进行路由决策；我们把基于流量水平做路由决策的可能性推迟到 5.3 节介绍。

5.2.1 优化原则

在讨论具体的算法之前，我们先不考虑网络拓扑结构和流量情况，而是给出最优路径的一般性论述，这有助于后续学习。这个论述称为最优优化原则（optimality principle）（Bellman, 1957）。最优路径的一般陈述如下：如果路由器 J 在从路由器 I 到路由器 K 的最优路径上，那么从 J 到 K 的最优路径也必定遵循同样的路由。为了更好地理解这点，我们将从 I 到 J 的路径部分记作 r_1 ，余下的路径部分记作 r_2 。如果从 J 到 K 还存在一条路由比 r_2 更好，那么，它可以与 r_1 级联起来，从而可以改善从 I 到 K 的路由，这与 $r_1 r_2$ 是最优路径的假设相违背。

作为最优优化原则的一个直接结果，从所有的源到一个指定目标的最优路径的集合构成了一棵以目标节点为根的树。这样的树称为汇集树（sink tree），如图 5-6（b）所示，图中的距离度量是跳数。所有路由算法的目标是为所有路由器找到这样的汇集树，并根据汇集树来转发数据包。

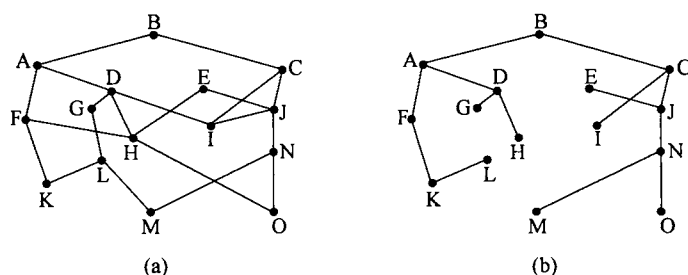


图 5-6
(a) 一个网络；(b) 路由器 B 的汇集树

请注意，汇集树不一定是唯一的；有可能存在具有相同路径长度的其他汇集树。如果我们允许选择所有可能的路径，则树就变成了更一般的结构，称为有向无环图（DAG，Directed Acyclic Graph）。DAG 没有环路。我们将把汇集树用作两种情况下的便利速记。两种情况都取决于路径不会互相干扰这样的技术假设，因此一条路径上的流量拥堵不会造成另一条路径的流量波动。

由于汇集树确实是一棵树，它不包含任何环，所以每个数据包将在有限的跳数内完成传递。然而，实际情形并非如此简单。在运行过程中，链路和路由器可能会故障，然后又恢复运行。所以，不同的路由器对当前拓扑结构的了解可能有所不同。而且，我们必须悄悄做出决定：每台路由器是独立地获取用于计算汇集树的信息，还是通过其他的方法来收集这些信息。稍后我们将很快回到这个问题。不管怎么样，最优优化原则和汇集树为测量其他路由算法提供了一个基准。

5.2.2 最短路径算法

现在我们从一个简单技术开始学习路由算法。给定一个完整的网络视图，这项技术可以用来计算最优路径。这些路径是那些我们希望分布式路由算法发现的，即使不是全部路由器都知道网络的所有细节。

基本想法是构造一张网络图，图中的每个节点代表一个路由器，每条边代表一条通信线路或者链路。为了选择一对给定路由器之间的路由，算法只需要在图中找出它们之间的最短路径。

最短路径（shortest path）概念值得做一些解释。一种测量路径长度的方法是跳数。若采用这种度量方法，则图 5-7 中 ABC 和 ABE 两条路径是等长的。另一种度量是以千米为单位的地理距离，在这种情况下，ABC 明显比 ABE 长很多（假定该图是按照比例画的）。

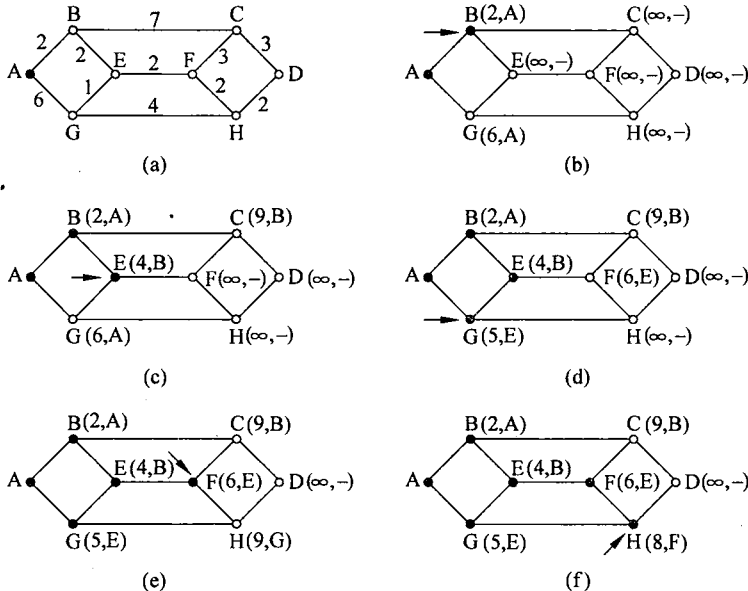


图 5-7 计算从 A 到 D 的最短路径的前 6 个步骤
箭头指向工作节点

然而，除跳数和物理距离之外，还可以用许多其他度量来标示路径的长短。例如，图中每条边用一个标准测试包的平均延迟来标记，这是每小时的测量结果。如果采用这种标记方法，则最短路径就是最快路径，而不是边数最少或者距离最短的路径。

一般情况下，边上面的标记可以作为距离、带宽、平均流量、通信成本、平均延迟等其他因素的一个函数，通过计算得出。通过改变函数的权重，路由算法就可以根据任何一种标准或者多种标准的组合来计算“最短”路径。

给定一个图，计算两个节点之间最短路径的算法已经有几个。这要归功于（Dijkstra, 1959）设计的一种算法，这种算法能找出网络中一个源节点到全部目标节点的最短路径。每一个节点都标出了（在圆括号内）从源节点沿着已知的最佳路径到达本节点的距离。距离必须是非负的，如果基于带宽和延迟这样的实际测量值，那么距离本来就不可能为负。初始时，所有的路径都不知道，因此所有节点都被标记为无限远。随着算法的不断进行，陆续有一些路径被找到，于是节点的标记可能发生变化，以便反映出更好的路径。每个标记可能是暂时的，也可能是永久的。初始时，所有的标记都是暂时的。当发现一个标记代表了从源节点到该节点的最短可能路径，该标记就变成永久，以后不再改变。

为了说明标记算法的工作过程，请看图 5-7（a）中的加权无向图，这里的权值代表了一种度量，比如说距离。我们现在要找到从 A 到 D 的最短路径。开始时，将节点 A 标记

为永久，在图中用一个实心圆表示；然后我们依次检查每一个与 A（工作节点）相邻的节点，并且用它们与 A 之间的距离重新进行标记。为了可以重构出最终路径，每当一个节点被重新标记时，也要标记出这次探测动作的出发节点（即前一个节点）。

在检查了每一个与 A 相邻的节点之后，我们检查整个图中全部具有暂时性标记的节点，并且使得其中具有最小标记的那个节点成为永久性的，如图 5-7（b）所示。这个节点就变成新的工作节点。

现在我们从 B 开始，检查所有与 B 相邻的节点。对于每一个与 B 相邻的节点，如果节点 B 上的标记加上从 B 到该节点的距离小于该节点原来的标记，说明我们找到了一条更短的路径，所以需要重新标记该节点。

在检查了所有与工作节点相邻的节点，并且可能时修改了暂时性标记之后，算法需要对整个图进行搜索，找到具有最小标记值的暂时性节点。这个节点的标记将变成永久性，并且成为下一轮的工作节点。图 5-7 显示了算法的前 6 个步骤。

为了看清楚该算法的工作原理，请看图 5-7（c）。此时，我们刚刚把 E 变成永久性节点。假设存在一条比 ABE 还要短的路径，比如说 AXYZE（对某个 X 和 Y），则有两种可能性：节点 Z 已经是永久性的，或者它还没有变成永久性节点。如果 Z 是永久性的，则 E 已经被探测过了（当 Z 变成永久性节点之后的下一轮探测），所以路径 AXYZE 不会逃脱我们的搜索范围，因而它不可能是一条最短路径。

现在考虑 Z 仍然是暂时性标记的情形。如果节点 Z 上的标记大于或者等于 E 上的标记，则 AXYZE 不可能是一条比 ABE 更短的路径；如果 Z 上的标记小于 E 上的标记，则应该首先是 Z 而不是 E 变成永久性节点，允许从 Z 探测 E。

图 5-8 给出了这个算法的描述。全局变量 n 和 dist 描述了图，在 shortest_path 被调用之前对这两个变量初始化。这段程序与前面描述的算法之间的唯一区别是，在图 5-8 所示的程序中，我们从终结节点 t 开始计算最短路径，而不是从源节点 s 开始。

```
#define MAX_NODES 1024          /* maximum number of nodes */
#define INFINITY 1000000000     /* a number larger than every maximum path */
int n, dist[MAX_NODES][MAX_NODES]; /* dist[i][j] is the distance from i to j */
void shortest_path(int s, int t, int path[])
{
    struct state{                /* the path being worked on */
        int predecessor;        /* previous node */
        int length;             /* length from source to this node */
        enum {permanent, tentative} label; /* label state */
    } state[MAX_NODES];
    int i, k, min;
    struct state *p;
    for (p = &state[0]; p < &state[n]; p++){ /* initialize state */
        p->predecessor = -1;
        p->length = INFINITY;
        p->label = tentative;
    }
    state[t].length = 0; state[t].label = permanent;
```

图 5-8 用图来计算最短路径的 Dijkstra 算法

```

k = t;                                /* k is the initial working node */
do{                                    /* Is there a better path from k? */
    for (i = 0; i < n; i++)            /* this graph has n nodes */
        if (dist[k][i] != 0 && state[i].label == tentative) {
            if (state[k].length + dist[k][i] < state[i].length) {
                state[i].predecessor = k;
                state[i].length = state[k].length + dist[k][i];
            }
        }
    /* Find the tentatively labeled node with the smallest label. */
    k = 0; min = INFINITY;
    for (i = 0; i < n; i++)
        if (state[i].label == tentative && state[i].length < min) {
            min = state[i].length;
            k = i;
        }
    state[k].label = permanent;
} while (k != s);
/* Copy the path into the output array. */
i = 0; k = s;
do {path[i++] = k; k = state[k].predecessor; } while (k >= 0);
}

```

图 5-8（续）

由于在一个无向图中，从 t 到 s 的最短路径与从 s 到 t 的最短路径是相同的，所以，无论从哪一个节点开始计算其实并没有多大关系。之所以选择后向搜索的原因是，每个节点都被标记了它的前任节点而不是它的继任节点。因此，当最终的路径被复制到输出变量 `path` 中的时候，该路径也被逆转了过来。两次逆转的效果刚好相互抵消，因此，结果按正确的顺序产生。

5.2.3 泛洪算法

在实现路由算法时，每个路由器必须根据本地知识而不是网络的全貌做决策。一个简单的本地技术是泛洪（flooding），这种技术将每一个入境数据包发送到除了该数据包到达的那条线路以外的每条出境线路。

很显然，泛洪法会产生大量的重复数据包。事实上，除非采取某些措施来抑制泛洪过程，否则将会产生无限多的数据包。其中一项措施是在每个数据包的头中设置一个跳计数器，每经过一跳该计数器减一，当计数器到达 0 时就丢弃该数据包。理想情况下，跳计数器的初始值应该等于从源端到接收方之间路径的长度。如果发送方不知道该路径有多长，它可以将计数器的初始值设置为最坏情形下的长度，即网络的直径。

带有跳计数器的泛洪能够产生随着跳数增大而指数增长的重复数据包，而且路由器要复制以前已经看到过的数据包。抑制数据包泛滥的一种更好技术是让路由器跟踪已经泛洪过的数据包，从而避免第二次发送它们。实现这个目标的一种方式是让每个源路由器在接收来自主机的数据包时填上一个序号，然后每个路由器为每个源路由器准备一张表，记录

已经观察到的来自源路由器的序号。如果入境数据包在这张表中，它就不能再被泛洪到其他路由器。

为了防止该表无限地膨胀，每个表应该使用一个计数器 k 作为参数，它表示直到 k 的所有序号都已经观察到了。当一个数据包抵达时，很容易检查该数据包是否已经被泛洪过（只需要比较该数据包的序号和 k ）；如果是被泛洪过，则丢弃该数据包。而且，不需要记录 k 以下的整个列表，因为 k 本身就是这部分数据的有效概括。

对于发送大多数数据包来说，泛洪算法是不切实际的，但它确实有一些重要的用途。首先，它确保数据包能被传送到网络中的每个节点。如果数据包的目的地只有一个，这种发送途径是一种浪费；但对于广播信息来说，这是一种有效的广播手段。在无线网络中，站传送的全部信息都可以被位于其无线范围内的所有其他站接收，这实际上就是一种形式的泛洪，一些无线路由算法利用了这个特性。

其次，泛洪途径的鲁棒性非常好。即使大量路由器被炸成碎片（例如，位于战争地区的一个军事网络），泛洪也能找到一条路径（如果存在），使得数据包到达目的地。泛洪需要的安装很少，路由器仅仅需要知道自己的邻居即可。这意味着，泛洪可以作为其他路由算法的基本构件，那些算法更有效但需要更多的处理。泛洪还可用作其他路由算法进行比较的性能度量。因为泛洪能并发选择每一条可能的路径，因此总能选出最短的那条路径，没有其他算法能够产生一个更短的延迟（如果我们忽略泛洪过程本身产生的开销）。

5.2.4 距离矢量算法

计算机网络通常使用动态路由算法。虽然动态路由算法比上述泛洪算法更复杂，但由于这些算法能找到当前网络拓扑中的最短路径，因而更加有效。特别地，两个动态算法最为流行，即距离矢量路由算法和链路状态路由算法。在这一小节中，我们考查前一个距离矢量路由算法；在下一小节，我们再讨论链路状态路由算法。

距离矢量路由（distance vector routing）算法是这样工作的：每个路由器维护一张表（即一个矢量），表中列出了当前已知的到每个目标的最佳距离，以及所使用的链路。这些表通过邻居之间相互交换信息而不断被更新，最终每个路由器都了解到达每个目的地的最佳链路。

距离矢量路由算法有时候也被叫做其他名称，最为常见是分布式 Bellman-Ford 路由算法，这种叫法是根据其设计者的名字来命名的（Bellman, 1957; Ford 和 Fulkerson, 1962）。距离矢量路由算法是最初 ARPANET 使用的路由算法，也曾被用于 Internet，相应的协议名称为 RIP 协议。

在距离矢量路由算法中，每个路由器维护一张路由表，它以网络每个路由器为索引，并且每个路由器对应一个表项。该表项包含两部分：到达该目标路由器的首选出境线路，以及到达该目标路由器的距离估计值。距离的度量可能是跳数，或者其他因素，正如我们在计算最短路径时讨论的那样。

假定路由器知道它到每一个邻居的“距离”。如果所用的度量是跳数，那么该距离就是 1 跳。如果度量值为传播延迟，则路由器很容易测量出链路的传播延迟。它只要直接发送一个特殊的 ECHO 数据包给邻居，邻居收到后盖上时间戳，尽可能快地发回来即可。

举个例子，假设使用延迟作为距离度量，并且路由器知道它到每个邻居的延迟。每隔

T 毫秒，每个路由器向它的每个邻居发送一个列表，该表包含了它到每个目标的延迟估计值；同时，它也从每个邻居那里接收到一个类似的表。想象一个路由器接收了来自邻居 X 的一个表，其中 X_i 表示邻居 X 估计的到达路由器 i 所需要的时间。如果该路由器知道它到邻居 X 的延迟为 m 毫秒，那么它也能明白在 X_i+m 毫秒之内经过 X 可以到达路由器 i。对每个邻居都执行这样的计算，最终可以发现到每个目标的最佳估计值，然后在新的路由表中使用这个最佳估计值以及对应的出境线路。请注意，在上述到目标距离的计算过程中没有使用老的路由表。

这个更新过程如图 5-9 所示。其中图 5-9 (a) 显示了一个网络。图 5-9 (b) 的前 4 列显示了 J 从邻居路由器接收到的延迟矢量。A 声称它到 B 有 12 毫秒的延迟，到 C 有 25 毫秒的延迟，到 D 有 40 毫秒的延迟，等等。假定 J 已经测量和估计了它到邻居 A、I、H 和 K 的延迟分别为 8、10、12 和 6 毫秒。

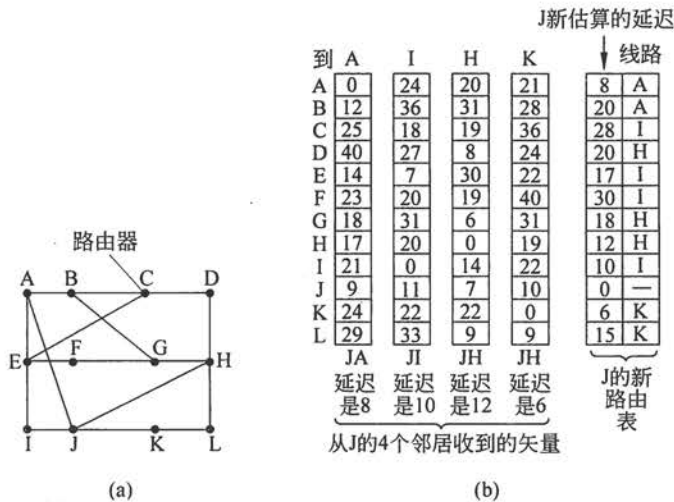


图 5-9 (a) 一个网络示例； (b) 来自 A、I、H、K 的输入，以及 J 的新路由表

现在考虑 J 如何计算它到路由器 G 的新路径。它知道在 8 毫秒之内可以到达 A，并且 A 声称可以在 18 毫秒内到达 G，所以 J 知道，如果它将那些目标地址为 G 的数据包转发给 A 的话，那么到 G 的延迟为 26 毫秒。类似地，它计算出经过 I、H 和 K 到达 G 的延迟分别为 41（即 31+10）、18（即 6+12）和 37（即 31+6）毫秒。在这些计算得出的距离值之中，最好的结果是 18，所以在 J 的路由表中，对应于 G 的表项中的延迟值为 18 毫秒，所用的路径是经过 H 的那条。对于所有其他的目标地址执行同样的计算过程，最后得到的新路由表如图中最后一列所示。

无穷计算问题

整个网络最佳路径的寻找过程称为收敛 (convergence)。距离矢量路由算法作为一项简单技术很有用，因为路由器可以在所有路径中有选择地计算出一条最短路径。但实际上它有一个严重的缺陷：虽然它总是能够收敛到正确的答案，但速度可能非常慢。尤其是，它对于好消息的反应非常迅速，而对于坏消息的反应异常迟缓。考虑这样一个路由器，它到目标 X 的最佳路径非常大。如果在下一次交换信息时，邻居 A 突然报告说它到 X 有一条

延迟很短的路径，那么，该路由器只是将发送给 X 的流量切换到通向 A 的线路。经过一次矢量交换，好消息就发挥作用了。

为了看清楚好消息的传播有多快，请考虑图 5-10 中一个五节点（线型）网络，这里的延迟度量为跳数。假定 A 最初处于停机状态，所有其他的路由器都知道这一点。换句话说，它们都将到 A 的延迟记录为无穷大。

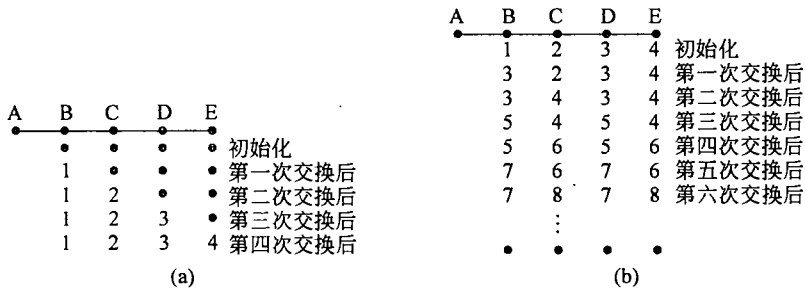


图 5-10 无穷计算问题

当 A 启动时，其他的路由器通过矢量交换知道了这一点。为了简化起见，我们假定在某个地方有个巨大的锣鼓，它定期敲击来启动所有路由器同时进行矢量交换。在第一次交换时，B 知道它左边的邻居到 A 的延迟为 0。于是 B 在它的路由表中建立一个表项，说明 A 离它一跳远。所有其他的路由器仍然认为 A 是停机的。这时候，针对 A 的路由表项如图 5-10 (a) 中的第二行所示。在接下去的交换中，C 知道 B 有一条路径通向 A，并且路径长度为 1，所以它更新自己的路由表，指明它到 A 的路径长度为 2，但是 D 和 E 要到以后才能听到这个好消息。很显然，好消息扩散的速度是每交换一次往远处走一跳。如果一个网络中最长路径是 N 跳，那么经过 N 次交换之后，每个路由器都将知道新恢复的链路和路由器。

现在我们考虑图 5-10 (b) 的情形，在这里所有的线路和路由器最初都是正常工作的。路由器 B、C、D 和 E 到 A 的距离分别为 1、2、3 和 4。突然间 A 停机了，或者 A 和 B 之间的链路断了（从 B 的角度来看这两种情况的效果是相同的）。

在第一次信息交换时，B 没有听到来自 A 的任何信息。幸运的是，C 说“别担心，我有一条通向 A 的长度为 2 的路径”。B 并不怀疑 C 的路径经过自身，B 能知道的全部只是 C 可能有 10 条链路，每条都独立地通向 A，并且长度为 2。因此，B 认为它可以通过 C 到达 A，路径长度为 3。在第一次交换之后，D 和 E 并不更新它们的 A 表项。

在第二次信息交换时，C 注意到，它的每一个邻居都声称有一条通向 A 的长度为 3 的路径。它随机地挑选出一条，并且将它到 A 的距离更新为 4，如图 5-10 (b) 中的第三行所示。通过后续的交换，可以得到图 5-10 (b) 中余下的记录历史。

从这个图中，我们应该很清楚地看出为什么坏消息传播得很慢：没有一个路由器具有一个比它所有邻居的最小值还大于 1 的值。逐渐地，所有的路由器都会趋向无穷大，但是所需交换的次数依赖于代表无穷大的数值。由于这样的原因，明智的做法是将无穷大设置为最长的路径加 1。

并不完全出人意料，这个问题称为无穷计数（count-to-infinity）问题。已经有许多试图解决该问题的方法，例如，防止路由器向邻居返回一个从该邻居获得的最佳路径，这个方法称为带有染毒逆向的水平分裂法，在 RFC 1058 中对此进行了讨论。然而，这些启发

式工作尽管有丰富多彩的名称，但实际工作效果并不好。问题的核心在于当 X 告诉 Y 它有一条通往某个地方的路径，Y 无从知道自己是否已在这条路径上。

5.2.5 链路状态路由

1979 年以前 ARPANET 一直使用距离矢量路由算法，而在此之后则改为使用链路状态路由算法。导致距离适量算法退位的主要问题在于，当网络拓扑结构发生变化后距离矢量路由算法需要太长时间才能收敛到稳定状态（由于无穷计数问题）。因此，距离矢量路由算法被一个全新的算法所替代，该算法称为链路状态路由算法（link state routing）。今天，链路状态路由算法的变种算法——IS-IS 或者 OSPF 已经成为大型网络或 Internet 应用最为广泛的路由算法。

链路状态路由算法的设计思想非常简单，可以用五个部分加以描述。每一个路由器必须完成以下的事情，算法才能正常工作：

- (1) 发现它的邻居节点，并了解其网络地址。
- (2) 设置到每个邻居节点的距离或者成本度量值。
- (3) 构造一个包含所有刚刚获知的链路信息包。
- (4) 将这个包发送给所有其他的路由器，并接收来自所有其他路由器的信息包。
- (5) 计算出到每个其他路由器的最短路径。

实际上，算法将完整的拓扑结构分发给了每一个路由器。然后每个路由器运行 Dijkstra 算法就可以找出从本地到每一个其他路由器的最短路径。下面我们详细地考虑上述每一个步骤。

发现邻居

当一个路由器启动时，它的第一个任务是找出哪些路由器是它的邻居。为了实现这个目标，它只需在每一条点到点线路上发送一个特殊的 HELLO 数据包。线路另一端的路由器应该返回一个应答说明自己是谁。这些名字必须是全局唯一的，因为当一个远程路由器以后听到有三个路由器都能连接到 F 时，它必须能够确定这三个路由器所提到的 F 是同一个路由器 F。

当两个或者多个路由器通过一个广播链路连接（比如一个交换机、环或经典以太网），情形会稍微复杂一些。图 5-11（a）显示了一个广播 LAN 直接与三个路由器 A、C 和 F 连接的情形。如图所示，每个路由器都连接到一个或者多个其他的路由器上。

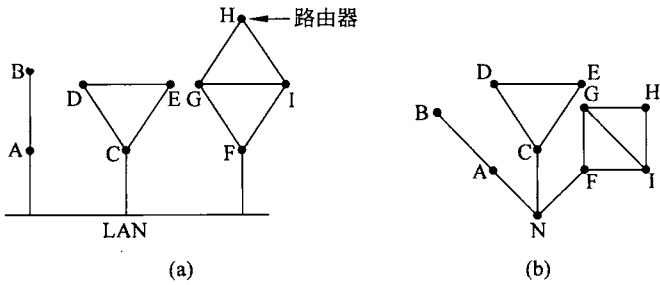


图 5-11
(a) 9 个路由器和一个广播 LAN；(b) 左侧网络的模型图

广播 LAN 为连接到其上的任何一对路由器提供了彼此的连通性。然而，把 LAN 建模成许多个点-to-点链路会增大拓扑结构，从而导致浪费消息。模拟局域网的一个更好方法来说是把它看作一个节点，如图 5-11 (b) 所示。在这里，我们引入了一个新的人造节点 N，它与 A、C 和 F 连接。LAN 上的一个指定路由器 (designated router) 被选中替代 N 运行路由协议。事实上，从 LAN 上的 A 到 C 是可能的，这里用路径 ANC 表示这条路径。

设置链路成本

为了寻找最短路径，链路状态路由算法需要每条链路以距离或成本度量。到邻居的成本可自动设置或由网络运营商配置的度量。一种常用的选择是使成本与链路带宽成反比。例如，1Gbps 以太网的成本可能是 1，而 100 Mbps 以太网的成本可能是 10。这样可以使得高容量的路径成为路由更好的选择。

如果网络在地理上分散，链路的延迟可以作为成本的组成部分，这样才能更好地选择较短链路上的路径。确定这种延迟的最直接方法是通过线路给另一边发送一个特殊的 ECHO 数据包，要求对方立即发回。通过测量往返时间再除以 2，发送路由器可以得到一个合理的延迟估算值。

构造链路状态包

一旦收集到了所需要的交换信息，每个路由器的下一步工作是构建一个包含所有这些信息的数据包。该数据包的内容首先是发送方的标识符，接着是一个序号 (Seq) 和年龄 (Age，后面再介绍)，以及一个邻居列表。对于每个邻居，同时要给出到这个邻居的延迟。图 5-12 (a) 显示了一个网络实例，每条线路上标出了延迟信息。这 6 个路由器所对应的链路状态数据包如图 5-12 (b) 所示。

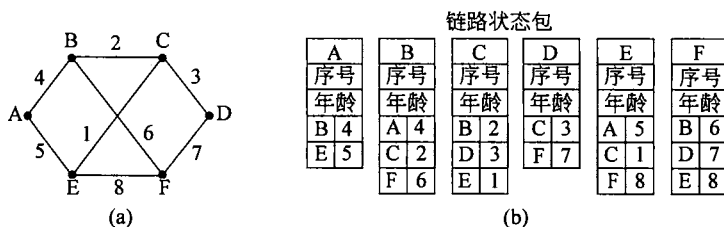


图 5-12

(a) 一个网络示例；(b) 该网络的链路状态包

构造链路状态数据包很容易。难的是确定什么时候构造数据包。一种可能的做法是周期性地创建数据包，也就是说，以一定的时间间隔创建链路状态数据包。另一种可能做法是每当发生某些重要的事情时才创建数据包，比如当一条线路断掉或者一个邻居节点停机时，或当它们重新恢复运行时，或当它们的特性发生了一定变化时。

分发链路状态包

链路状态路由算法最技巧的部分在于分发链路状态数据包。所有路由器必须快速并可靠地获得全部的链路状态数据包。如果不同的路由器使用了不同版本的拓扑结构，那么它们计算出来的路由可能会不一致，例如出现环路、目标机器不可达以及其他的问题。

首先，我们描述最基本的发布算法，然后再对它进行改进。基本思路是使用泛洪法将链路状态数据包分发给所有路由器。为了控制泛洪规模，每个数据包都包含一个序号，序

号随着每一个新数据包发出而逐一递增。路由器记录下它所看到的所有（源路由器、序号）对。当一个新的链路状态数据包到达时，路由器检查这个新来的数据包是否已经出现在上述观察到的列表中。如果这是一个新数据包，则把它转发到除入境线路之外的所有其他线路上。如果这是一个重复数据包，则将它丢弃。如果数据包的序号小于当前所看到过的来自该源路由器的最大序列号，则它将被当作过时数据包而拒绝接受，因为路由器已经有了更新的数据。

这个算法还有几个问题有待处理，不过，这些问题都是可管理的。第一，如果序号绕回，可能会产生混淆。这里的解决方案是使用一个 32 位的序号。即使每秒钟产生一个链路状态数据包，也需要 137 年才可能发生绕回，所以，这种可能性可以忽略不计。

其次，如果一个路由器崩溃了，那么它将丢失所有的序号记录表。如果它再从 0 开始，那么，下一个数据包将被作为重复数据包而拒绝。

再次，如果一个序号被破坏了，比如发送方发送的序号为 4，但是由于产生了 1 位错误，所以接收方看到的序号是 65 540，那么，序号从 5 到 65 540 的数据包都将被当作过时数据包而拒绝接受，因为接收方认为当前的序号是 65 540。

所有这些问题的解决方案都一样：在每个数据包的序号之后包含一个年龄（age）字段，并且每秒钟将年龄减 1。当年龄字段的值被减到 0 时，来自路由器的该信息将被丢弃。通常情况下，每隔一段时间，比如说 10 秒，一个新的数据包就会到来；所以，只有当一个路由器停机时（或者 6 个连续的数据包被丢失，这种情形发生的可能性不大），路由器信息才会超时。在初始泛洪过程中，每个路由器也要递减 age 字段，这样可以确保没有数据包丢失，也不会无限制生存下去（如果一个数据包的 age 为 0，则被丢弃）。

对这个算法做一些改进可以使它更加健壮。当一个链路状态数据包被泛洪到一个路由器时，它并没有立即被排入队列等待传输。相反，它首先被放到一个保留区中等待一段时间。如果在这个数据包被转发出去之前，另一个来自于同一个源路由器的链路状态数据包也到来了，那么就比较它们的序号。如果两个数据包的序号相等，则丢弃重复数据包。如果两者不相等，则丢弃老的数据包。为了防止线路产生错误导致丢包和错包，所有的链路状态数据包都要被确认。

在图 5-12（a）所示的网络中，路由器 B 使用的数据结构如图 5-13 所示。这里的每一行对应于一个刚刚到达的，但是还没有完全处理完毕的链路状态数据包。该表记录了数据包的来源、序号和年龄，以及状态数据。而且，针对 B 的三条线路（分别到 A、C 和 F）还记录了发送和确认标志。发送标志表明该数据包必须在所指示的线路上发送。确认标志表明它必须在这条线路上得到确认。

源路由器	序号	年龄	发送标志 ACK标志						数据
			A	C	F	A	C	F	
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

图 5-13 在图 5-12（a）中路由器 B 的状态包缓冲区

在图 5-13 中, 来自 A 的链路状态数据包可以直接到达, 所以 B 必须将它发送给 C 和 F, 并且按照标志位的指示向 A 发回确认。类似地, 必须把来自 F 的数据包转发给 A 和 C, 并且向 F 返回确认。

然而, 第三个数据包, 即来自 E 的数据包有所不同。它到达两次, 一次经过 EAB, 另一次经过 EFB。因此, 它只需被发送给 C, 但是要向 A 和 F 确认, 正如标志位所示。

如果一个重复数据包到来时原来的数据包仍然在缓冲区中, 那么标志位必须作相应的改变。例如, 如果表中第四项被转发出去之前, C 的链路状态数据包的一份副本从 F 到达, 那么, 这六位将被改变为 100011, 以表明该数据包必须向 F 确认, 但是不用转发了。

计算新路由

一旦路由器已经积累了全部的链路状态数据包之后, 它就可以构造出完整的网络图, 因为每条链路都已经被表示出来了。事实上, 每条链路被表示了两次, 每个方向各表示一次。不同方向的链路可能有不同的成本。最短路径计算可找到从 A 到 B 与从 B 到 A 不同的路径。

现在可以在路由器本地运行 Dijkstra 算法, 以便构建出从本地出发到所有可能目标的最短路径。这个算法的运行结果告诉路由器到达每个目的地能够走哪条链路。这个信息被安装在路由表中, 而且恢复正常操作。

相比距离向量算法, 链路状态路由算法需要更多的内存和计算。对于一个具有 n 个路由器的网络, 每个路由器有 k 个邻居, 那么, 用于存储输入数据所要求的内存与 kn 成正比, 这至少与列出全部目的地的路由表一样大。而且, 计算时间的增长快过 kn , 即使采用最有效的数据结构, 在大型网络中运行这个算法依然是个问题。不过, 在许多实际场合, 链路状态路由算法工作得很好, 因为它没有慢收敛问题。

链路状态路由算法被广泛地应用于实际网络中, 所以现在提一下某些使用该算法的例子。许多 ISP 使用中间系统到中间系统 (IS-IS, Intermediate System-Intermediate System) 链路状态协议 (Oran, 1999), 它是专门为 DECnet 而设计的, 后来被 ISO 采纳用于 OSI 协议; 自此以后, 它被作了多次修改以便能够处理其他的协议, 例如最著名的 IP 协议。开放最短路径优先 (OSPF, Open Shortest Path First) 是另一个主流链路状态协议。它是在 IS-IS 之后几年由 IETF 设计的, 而且它吸收了 IS-IS 的许多创意。这些创意包括: 一种泛洪链路状态更新的自稳定方法、LAN 上的指定路由器概念以及计算和支持路径分裂和多个度量的方法。因此, IS-IS 和 OSPF 之间的差异非常小。其中一个最重要的差别是, IS-IS 可同时携带多个网络层协议的信息 (比如, IP、IPX 和 AppleTalk), OSPF 不具备这个特性。对于大型的多协议环境这一特性是个优势。我们将在 5.6.6 节介绍 OSPF。

下面依次对路由算法做出一一般性的评论。连接状态、距离向量和其他路由算法依赖于所有路由器计算路径的处理。硬件或软件, 甚至少数路由器的的问题都可以肆虐破坏网络。例如, 如果一个路由器声称有一条实际上并不存在的链路, 或者忘记一条实际上存在的链路而声称没有, 由此产生的网络图都将是不正确的。如果一个路由器转发数据包失败, 或者在转发期间自己发生故障, 对应的路由将无法正常工作。最后, 如果内存不足或者路由计算出错, 就会发生不好的事情。随着网络规模几十、几百或者上千个节点的增长, 某些路由器偶尔会失败的概率变得不可忽视。应对这个困境的诀窍是尽量做好安排, 当发生不

可避免的错误时把危害降到最低。(Perlman, 1988) 详细讨论了这些问题并给出了可能的解决方案。

5.2.6 层次路由

随着网络规模的增长，路由器的路由表也成比例地增长。不断增长的路由表不仅消耗路由器内存，而且还需要更多的 CPU 时间来扫描路由表以及更多的带宽来发送有关的状态报告。当网络增长到一定时可能会达到某种程度，此时每个路由器不太可能再为其他每一个路由器维护一个表项。所以，路由不得不分层次进行，就好像电话网络中的做法那样。

在采用了分层路由之后，路由器被划分成区域 (region)。每个路由器知道如何将数据包路由到自己所在区域内的目标地址，但是对于其他区域的内部结构毫不知情。当不同的网络被相互连接在一起，很自然地就会将每个网络当作一个独立的区域，一个网络中的路由器不必知道其他网络的拓扑结构。

对于大型网络，两级的层次结构可能还不够；可能有必要将区域组织成簇 (cluster)，将簇组织成区 (zone)，将区组织成群 (group)，等等，直到将所有的集合名词用完为止。举一个多层结构的例子，请考虑如何将一个数据包从加利福尼亚州的伯克利路由到肯尼亚的马琳迪。伯克利的路由器知道加利福尼亚州的详细拓扑结构，但是可能将所有州际的流量发送给洛杉矶的路由器；洛杉矶的路由器能够将流量直接路由给美国其他的路由器，但是它会将国际流量发到纽约；纽约的路由器直接将所有的流量发送至目标国家中负责处理国际流量的路由器，比如肯尼亚的内罗毕。最后，该数据包将沿着肯尼亚国家中的路径树往下传送，一直到达马琳迪 (Malindi)。

图 5-14 给出了一个两级层次的定量分析例子，其中包含了 5 个区域。路由器 1A 的完整路由表有 17 个表项，如图 5-14 (b) 所示。如果采用分级路由，则路由表如图 5-14 (c) 所示，所有针对本区域内的路由器表项都跟原先一样；但是，所有到其他区域的路由都被压缩到了单个路由器中。因此，所有到区域 2 的流量都要经过 1B-2A 线路，其余的远程流

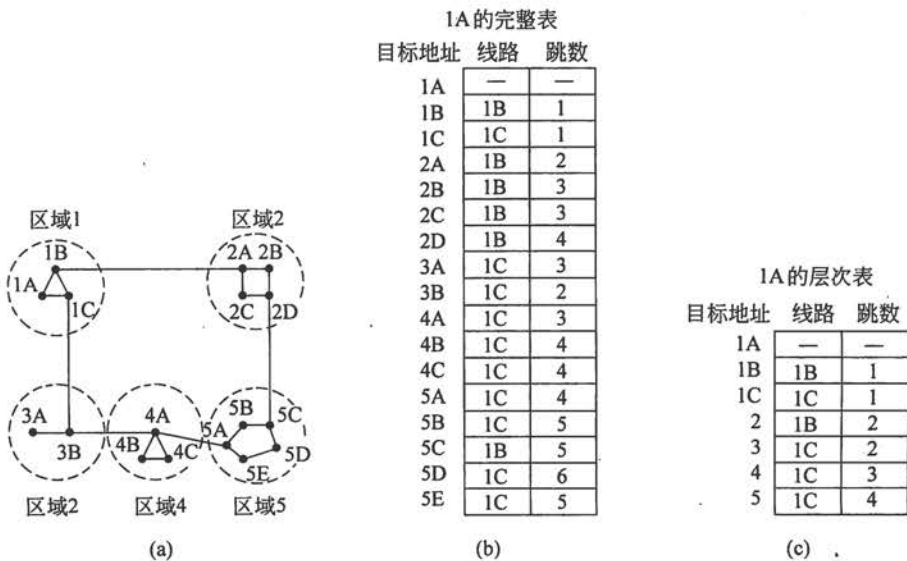


图 5-14 层次路由

量都经过 1C-3B 线路。层次路由使得路由器 1A 的路由表长度从 17 项降低为 7 项。随着区域数与每个区域中路由器数量之比值的增加,节省下来的表空间也随之增加。

不幸的是,这种空间的节省不是免费得来的,它需要付出代价,其代价的形式是增加了路径长度。例如,从 1A 到 5C 的最佳路径是经过区域 2,但采用了层次路由之后,所有到区域 5 的流量都要经过区域 3,因为对于区域 5 中的大多数目标来说这是更好的选择。

当单个网络变得非常大时,一个有趣的问题是“应该分多少层?”。例如,考虑一个具有 720 个路由器的子网。如果没有分层,每个路由器需要 720 个路由表项;如果子网被分成 24 个区域,每个区域 30 个路由器,那么每个路由器只需要 30 个本地表项,加上 23 个远程表项,总共 53 个表项;如果采用三级层次结构,总共 8 个簇,每个簇包含 9 个区域,每个区域 10 个路由器,那么,每个路由器需要 10 个表项用于记录本地路由器,8 个表项用于到同一簇内其他区域的路由,7 个表项用于远程的簇,总共 25 个表项。(Kamoun 和 Kleinrock, 1979) 发现,对于一个包含 N 个路由器的网络,最优的层数是 $\ln N$, 每个路由器所需的路由器表项是 $e \ln N$ 个。他们还证明了,由于分层路由而导致的平均路径长度的实际增长非常小,通常是可以接受的。

5.2.7 广播路由

在有些应用中,主机需要给其他多个或者全部主机发送消息。例如,用于发布天气预报、股市行情最新报告或者现场直播节目等服务,它们的最佳工作方式是将消息广播给所有的机器,然后让那些感兴趣的机器读取数据。同时给全部目标地址发送一个数据包称为广播 (broadcasting); 为了实现广播,人们已经提出了各种各样的方法。

一种不要求网络具有任何特殊性质的广播方法是让源机器简单地给每一个目标单独发送一个数据包。这种方法不仅浪费带宽,而且还要求源机器拥有所有目标机器的完整地址列表。实际上这种做法不够理想,即使它广泛适用。

一种改进方法称为多目标路由 (multidestination routing), 每个数据包包含一组目标地址, 或者一个位图, 由该位图指定所期望到达的目标。当一个数据包到达一个路由器时, 路由器检查数据包携带的所有目标, 确定哪些输出线路是必要的 (只要一条输出线路是到达至少一个目标的最佳路径, 那么它就是必要的)。路由器为每一条需要用到的输出线路生成一份该数据包新的副本, 在这份副本中只包含那些使用这条线路的目标地址。实际上, 原来的目标集合被分散到这些输出线路上。在经过了足够多的跳数之后, 每个数据包将只包含一个目标地址, 因此可以被当作普通的数据包来对待。多目标路由方法就如同单个地址的数据包一样, 只不过当多个数据包必须遵循同样的路径时, 其中一个数据包承担了全部的费用, 而其他的数据包则是免费搭载。因此, 网络带宽的利用率更高。然而, 这种模式依然要求源端知道全部的目标地址, 对于路由器来说, 要确定从哪些线路转发多目标数据包的工作量太大, 尤其是处理多个不同的数据包时。

我们早就看到了更好的广播路由技术——泛洪。当每个源实现了序号, 泛洪能有效地利用链路, 而且路由器要做的决策相对地简单。虽然泛洪方法不适合普通的点到点通信, 但它被认为值得考虑用作广播。然而, 事实证明, 一旦计算出普通数据包的最短路径, 我们可以把广播做得更好。

逆向路径转发 (reverse path forwarding) 思想一经提出就获得了关注, 它被认为是一种既优雅又相当简单的广播技术。当一个广播数据包到达一个路由器时, 路由器检查它到来的那条线路是否正是通常用来给广播源端发送数据包用的那条线路。如果是, 说明这是一个极好的机会, 该广播数据包是沿着最佳路径被转发过来的, 因而是到达当前路由器的第一份副本。如果是这种情况, 则路由器将该数据包转发到除了到来的那条线路之外的所有其他线路上。然而, 如果广播数据包是从其他任何一条并非首选的到达广播源的线路入境的话, 该数据包被当作一个可能的重复数据包而丢弃。

逆向路径转发算法的一个例子如图 5-15 所示。图 5-15 (a) 部分显示了一个网络, 图 5-15 (b) 部分显示了该网络中路由器 I 的一棵汇集树, 图 5-15 (c) 部分显示了逆向路径算法是如何工作的。在第一跳, I 发送数据包给 F、H、J 和 N, 如树中第二行所示。这些数据包中的每一个都是在通向 I 的首选路径 (假定首选的路径都沿着汇集树) 到来的, 这点用字母外面加一个圆圈来表示。在第二跳, 共产生了 8 个数据包, 其中, 在第一跳接收到数据包的路由器各产生 2 个数据包。结果, 所有这 8 个数据包都到达了以前没有访问过的路由器, 其中 5 个是沿着首选线路到来的。在第三跳所产生的 6 个数据包中, 只有 3 个是沿首选路径 (在 C、E 和 K) 到来的, 其他的都是重复数据包。在经过五跳和 24 个数据包以后, 广播过程终止。相比之下, 如果完全沿着汇集树的话, 只需要 4 跳和 14 个数据包。

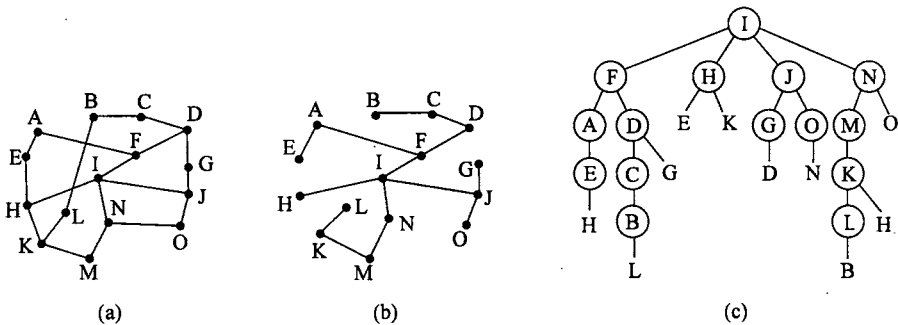


图 5-15 逆向路径转发

(a) 一个网络示例; (b) 一棵汇集树; (c) 由逆向路径转发构成的树

逆向路径转发的主要优点是它有效而且易于实现。它只往每个方向上的链路发送一次广播数据包, 就像泛洪一样简单, 而仅仅要求路由器知道如何到达全部目标; 路由器无须记住序号 (或使用其他机制来停止泛洪) 或者在数据包中列出全部的目标地址。

最后一种算法改进了逆向路径转发的行为。它明确使用了以发起广播的路由器为根的汇集树, 或者任何其他便利的生成树。**生成树 (spanning tree)** 是网络的一个子集, 它包含所有的路由器, 但是没有任何环路。汇集树是生成树的一种。如果每个路由器都知道它的哪些线路属于生成树, 那么, 它就可以将一个入境广播数据包复制到除了该数据包到来的那条线路之外的所有生成树线路上。这种方法可最佳使用带宽, 并且所生成的数据包也绝对是完成这项任务所需要的最少数量。例如, 图 5-15 (b) 的汇集树就被用作生成树, 广播数据包的副本最少, 只有 14 个。唯一的问题是, 每个路由器都必须知道这棵生成树才可以正常工作。有时候这样的信息是可以得到的 (比如采用了链路状态路由算法, 所有路由器都知道完整的网络拓扑, 因而它们可以计算出一棵生成树), 但是有时候无法获得这样的信

息（比如采用了距离矢量路由算法）。

5.2.8 组播路由

有些应用，比如多人游戏或者体育赛事视频直播到几个观看点，这样的应用将数据包发送给多个接收者。除非组的规模很小，否则给每个接收者单独发不同的数据包的代价很昂贵。另一方面，如果在一个由百万节点组成的网络中有一个由 1000 个机器组成的组，采用广播技术发送数据包显然也是一种浪费，因为大多数接收者对广播的消息并不感兴趣（甚至最糟糕的是他们虽然感兴趣，但不应该看到这些消息）。因此，我们需要有一种办法能够给明确定义的组发送消息，这些组的成员数量虽然很多，但相比整个网络规模却很小。

给这样的一个组发送消息称为组播（multicasting），使用的路由算法称为组播路由（multicast routing）。所有的组播方案都需要一些方法来创建和撤销特定的组，并确定哪些路由器是组的成员。如何完成这些任务不是路由算法要关注的。现在，我们假定每个组由一个组播地址标识，并且路由器知道自己属于哪些组。我们在 5.6 节讨论 Internet 网络层时再重温组成员的有关内容。

组播路由方案建立在我们已经学习过的广播路由方案的基础上，为了将数据包传递给组的成员同时又有效利用带宽，数据包可沿着生成树发送。然而，最佳生成树的使用取决于组的密度分布。密集分布指接收者遍布在网络的大部分区域；稀疏分布指大部分网络都不属于组。在本节，我们考虑这两种情况。

如果组的分布是密集的，那么广播是一个良好的开端，因为它能有效地把数据包发到网络的每个角落。但广播可能将到达一些不属于该组成员的路由器，因而也是一种浪费。（Deering 和 Cheriton, 1990）探索出一个解决方案，就是通过修剪广播生成树把不通往组成员的链路从树中删掉。修剪结果得到的是一棵有效的组播生成树。

作为一个例子，考虑如图 5-16 (a) 网络，其中有两个组：组 1 和组 2。有些路由器连接的主机属于其中的一个组或同时属于两个组，如图所示。最左边的路由器的一棵生成树如图 5-16 (b) 所示。此树可用于广播，对于组播来说则过度了，这从下面显示的两个修剪版本可见一斑。在图 5-16 (c) 中，所有不通往组 1 成员的主机的链路已被删除，结果是一棵针对最左边路由器发送到组 1 的生成树。数据包的转发只能沿着这棵树进行，可见这比广播树有效，因为这里只有 7 条链路而不是 10 条链路。图 5-16 (d) 显示了一棵针对组 2 修剪后的组播生成树。相比广播树它也更加有效，此时只有这 5 条链路。这个例子表明，不同的组播组有不同的生成树。

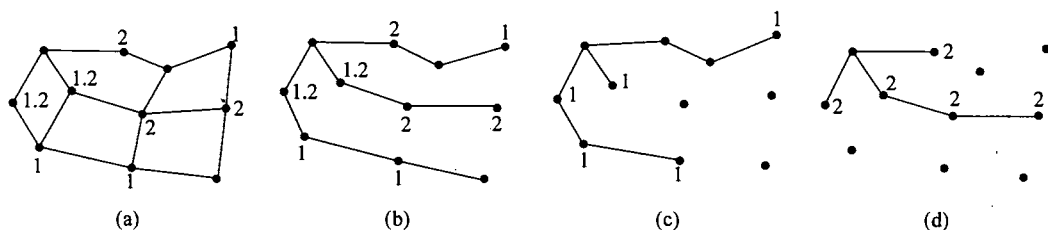


图 5-16

(a) 一个网络示例；(b) 最左侧路由器的一棵汇集树；(c) 组 1 的一棵组播树；(d) 组 2 的一棵组播树

生成树的修剪方式有许多种。如果路由器使用了链接状态路由算法，并且每个路由器知道完整的网络拓扑结构，包括了解哪些主机属于哪个组，那么可以使用一种最简单组播算法。每个路由器针对组内的每个发送者构造一棵它自己的修剪后生成树，具体做法是先按常规方法构造一棵以发送者为根的汇集树，然后从汇集节点上删除所有不连到组成员的链路。组播（MOSPF, Multicast OSPF）就是一个以这种方式工作的链路状态协议例子（Moy, 1994）。

如果采用距离矢量路由算法，则要遵循不同的修剪策略。基本算法是逆向路径转发。然而，一旦一个路由器没有任何主机对某个组感兴趣，并且没有连接到需要接收该组播消息的其他路由器，那么它要用 PRUNE 消息作为接收组播消息的响应，告诉发送该消息的邻居不要再给自己发送任何来自该组发送者的消息。如果一个路由器自己所连的主机没有一个属于该组成员，并且从它以前转发组播消息的所有线路都接收了这样的修剪消息，那么它也同样以 PRUNE 消息来响应。通过这种递归方式，最终修剪出一棵生成树。距离矢量组播路由协议（DVMRP, Distance Vector Multicast Routing Protocol）就是一个以这种方式工作的组播路由协议例子（Waitzman 等, 1988）。

修剪过程的最后结果得到的是一棵有效生成树，该树只用到那些抵达组成员真正需要的链路。这种方法的一个潜在缺点是路由器需要做大量的工作，特别是大型网络。假设一个网络有 n 个组，每个组平均有 m 个节点。在每个路由器上针对 n 组，每个组有 m 棵修剪生成树，因此总共需要存储 mn 棵生成树。例如，图 5-16 (c) 给出了最左边路由器给组 1 成员发送所用的生成树。最右边的路由器给组 1 发送数据包所用的生成树（未显示）将完全不同，因为组播数据包直接朝着组成员去而不会通过图的左侧发送。反过来，这意味着，路由器转发数据包将沿着不同的方向进行，具体方向取决于组内哪个节点是发送者。当存在大量的组，并且组内发送者很多时，需要大量的空间来存储所有的树。

另一种设计是采用基于核心树（core-based trees）的技术，计算某个组的单棵生成树（Ballardie 等, 1993）。采用这种方法时所有路由器都同意某个路由器作为根，这个根称为核心（core）或会聚点（rendezvous point），然后每个成员通过给根发送一个数据包来建立这棵树。树是组播数据包遵循的路径集合。图 5-17 (a) 显示了一棵组 1 的核心树。为了把数据包发送到这个组，发送者把数据包发给核心；当数据包到达核心后，它再被沿着树往下转发。图 5-17 (b) 显示了网络右侧一个发送者的组播过程。作为性能优化的一种措施，发往该组的数据包并不需要先发送到核心然后再开始组播。一旦数据包到达树，它便沿着树向上转发给根，但同时沿着树转发到其他分枝。这种情况由 5-17 (b) 中的右上角发送者显示。

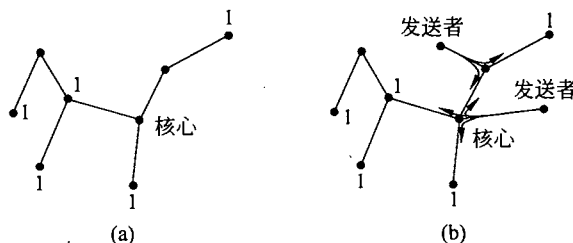


图 5-17

(a) 组 1 的基于核心树；(b) 给组 1 成员发送数据包

对于所有的组播源使用同一棵共享树是无法达到最优的。例如，在图 5-17 (b) 中，从网络右侧的发送者到达右上方的组成员通过核心要三跳，如果直接发送或许不需要三跳。共享树的低效率取决于核心和发送者的位置，把核心设置在所有发送者的中间往往是一种合理的做法。如果只有一个发送者，比如视频流传输到一个组，那么将发送者作为核心是最优的。

另外值得注意的是共享树可以大大节省存储开销、消息发送和计算。每个路由器只需要为每个组保存一棵树，而不是 m 棵树。此外，不属于这棵共享树一部分的路由器根本不需要为组做任何工作。正是出于这个原因，像这类基于核心树的共享树方法被用于 Internet 的稀疏组播，成为流行协议的一部分，例如协议独立组播 (PIM, Protocol Independent Multicast) (Fenner 等, 2006)。

5.2.9 选播路由

到目前为止，我们已经学习了三种数据包传递模型：源给单个目标节点发送（称为单播）、给所有目标节点发送（称为广播）以及给一组目标节点发送（称为组播）。实际上，还有另一种称为选播 (Anycast) 的传递模型有时也非常有用。在选播方式下，数据包被传递给最近的一个组成员 (Partridge 等, 1993)。发现这些路径的方案就是所谓的选播路由 (Anycast routing)。

我们为什么还要选播？有的时候，节点提供了诸如报时或者内容分发等服务，这类服务对客户而言最重要的是获得正确的信息而不是与哪个节点取得联系，任何节点都可以，只要它能提供所需的服务。例如，选播就作为域名系统的一部分广泛应用于 Internet 上，关于域名系统我们将在第 7 章讨论。

幸运的是，我们不需要为选播制定新的路由方案，因为普通的距离矢量和链路状态路由算法可用来生成选播路由。假设我们要选播数据包到组 1 的成员，该组成员都将被赋予一个组地址 “1” 而不是一个个独立的地址。距离矢量路由将像往常一样分发向量，并且节点将只选择到目的地 1 的最短路径。这将导致数据包被发送到目的地 1 的最近实例。图 5-18 (a) 显示了选播路由。此过程之所以能正常工作是因为路由协议并没有意识到目的地有多个实例。也就是说，它认为节点 1 的实例都是同一个节点，如图 5-18 (b) 所示的拓扑结构。

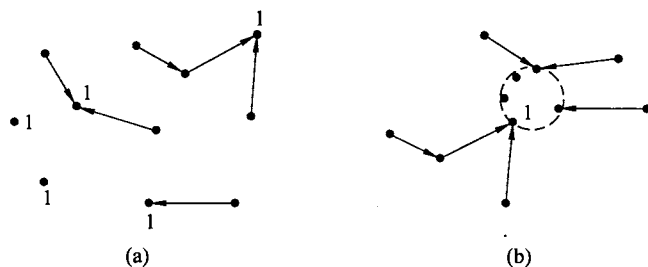


图 5-18

(a) 组 1 的选播路由； (b) 路由协议看到的拓扑结构

这个过程也同样适用于链路状态路由协议，虽然要额外考虑这样的情况，路由协议似

乎找不到通过节点 1 的最短路径。这将导致跨越网络空间的跳跃, 因为节点 1 的实例是那些真正位于网络不同部分的节点。然而, 链路状态协议已经能区分路由器和主机。我们首先掩盖了这一事实是因为我们没讨论它的必要。

5.2.10 移动主机路由

现在成千上万的人在移动的过程中使用计算机。比如坐在移动的汽车内使用无线设备, 这是一类真正的移动情形; 在一系列不同的位置使用笔记本电脑, 这是一类漫游的情形。我们将使用术语**移动主机** (mobile host) 示意上述任何一类场景中的设备, 以便与从不移动的固定主机截然区分。越来越多的人希望无论走到世界上任何地方都能保持联系, 就好像他们在家一样。这些移动主机引入了新的复杂性: 路由一个数据包到移动主机, 网络首先要找到该主机。

我们将考虑的模型世界中, 假设所有主机都有一个永久的家乡位置 (home location), 该位置永远不会改变。每个主机也有一个永久的家乡地址 (home address), 用来确定其家乡位置。家乡地址有点类似于电话号码, 比如 1-212-5551212 表明这个号码在美国 (国家代码 1) 的曼哈顿地区 (212)。移动主机所在系统的路由目标是使人们有可能利用固定的家乡地址来发送数据包, 无论他们在哪里都能有效地把数据包送到。当然, 这里的关键是要找到他们。

针对这种通信模型进行适当的讨论是有意义的。一种不同的模型是每当移动主机移动, 以及拓扑发生变化后就重新计算路由。然后我们可以简单地采用本节前面所述的路由方案。然而, 随着移动主机的数目越来越多, 这种模式将很快导致整个网络陷入不断地计算新路由。使用家乡地址能大大降低这种负荷。

另一种方法是在网络层之上提供移动, 这就是今天笔记本电脑典型的用法。当它们被转移到新的 Internet 位置, 笔记本电脑就获得一个新的网络地址。这里新老地址之间不存在任何关联; 网络也不知道它们属于同一台笔记本电脑。在此模型中, 一台笔记本电脑可用于浏览网页, 但其他主机无法给它发送数据包 (例如, 一个入境呼叫), 除非有更高层的位置服务, 例如移动之后再次登录 Skype。此外, 主机在移动期间无法保持与网络的连接, 而是必须重新启动建立新的连接。网络层的移动性对解决这些问题非常有用。

Internet 和蜂窝网络的移动路由采用的基本想法是移动主机把当前自己在哪里告诉给家乡位置的一台主机。这台主机称为**家乡代理** (home agent), 它将以移动主机的名义采取行动。一旦它知道移动主机的当前位置, 它就可以转发数据包以便数据包传递给移动主机。

图 5-19 显示了移动路由采取的动作。一个在西北部城市西雅图的发送者想发送一个数据包给通常设在美国纽约的主机。我们关心的一种情况是当移动主机不在家, 相反它暂时到了圣地亚哥, 怎么办?

在圣地亚哥的移动主机必须获得一个本地网络地址, 然后才能使用网络。这是主机获得网络地址通常会发生的情况, 我们将在本章后面讨论 Internet 时涉及这方面内容。本地地址称为**转交地址** (care of address)。一旦移动主机有了这个地址, 它可以告诉其家乡代理它现在哪儿。它给家乡代理发送一个带有转交地址的注册消息 (第 1 步)。该消息用图 5-19 中的虚线表示, 以表明它是一个控制信息而不是一个数据报文。

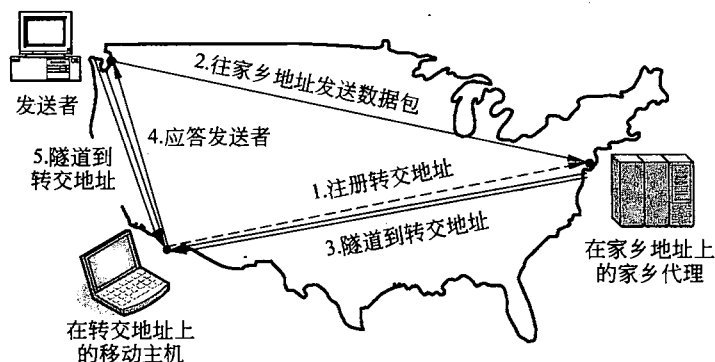


图 5-19 移动主机的数据包路由过程

接下来，发送者使用其永久地址发送一个数据包给移动主机（第 2 步）。这个数据包被网络路由到主机的家乡位置，因为这是主机家乡地址的所属地。在纽约，家乡代理截获该数据包，因为移动主机已经离家；然后用一个新的头包裹或者封装（encapsulated）该数据包，并把捆绑后的结果发送给转交地址（第 3 步）。这种机制称为隧道（tunneling）。该机制在 Internet 上非常重要，所以我们以后会更详细地考查它。

当封装后的数据包到达转交地址，移动主机解开它并提取来自发送者的数据包；然后移动主机直接给发送者发应答数据包（第 4 步）。整个路由过程称为三角路由（triangle routing），因为如果远程位置离家乡位置很远时，这条路由可能是迂回的。作为第 4 步的一部分，发送者可借鉴当前的转交地址，把随后的数据包直接发送给移动主机。具体做法是通过隧道发送到转交地址（第 5 步），完全绕过家乡位置。无论什么原因，在移动主机移动时如果丢失了连接，则家乡地址可随时用来寻址到移动主机。

我们在整个描述过程中忽略了一个很重要的方面，即安全性。一般来说，当一个主机或路由器得到这种形式的信息“现在开始，请把 Stephany 的所有邮件发送给我，”时，可能会产生两个问题，我在跟谁谈话，以及这是否是个好主意。安全信息被包含在消息中，因此可以用加密协议检查消息的合法性。我们将在第 8 章学习加密协议。

移动路由算法有许多变种。上面叙述的方案是 IPv6 流动模型，这种移动主要形式用在 Internet (Johnson 等, 2004)，以及诸如 UMTS 蜂窝网络中基于 IP 的那部分。我们发现如果发送者是个固定节点，则情况能简单些；但设计方案时必须考虑这两个节点都是移动的情形。另外，主机可能是移动网络的一部分，例如，在一个平面上的计算机。对基本方案进行扩展就可以支持移动网络，不涉及任何主机部分的工作 (Devarapalli 等, 2005)。

有些方案利用外地（即远程）代理，类似家乡代理但放置在远程位置，或类似于蜂窝网络中的访问位置寄存器 (VLR, Visitor Location Register)。然而，最近的研究方案并不需要外地代理；移动主机自己承担外地代理的行为。在这两种情况下，移动主机的临时位置仅限于被少量的主机获得（例如，移动主机、家乡代理和发送者），因此大型网络中的许多路由器不需要重新计算路由。

有关移动路由的更多信息，请参考 (Perkin, 1998, 2002)，以及 (Snoeren 和 Balakrishnan, 2000)。

5.2.11 自组织网络路由

我们现在已经看到当主机移动而路由器固定时,路由工作是如何进行的。另一种更极端的情形是路由器本身也是移动的。这种可能性主要发生在以下几种情形下:地震现场的紧急救援工作、战场上的军事车辆、海上航行的一队舰船或者一群配备了笔记本电脑的人们聚集在一个没有 802.11 网络的地区。

在所有这样或那样的情形中,每个节点用无线通信,并同时承担路由器和主机的双重角色。如果网络中的节点彼此靠近,那么该网络称为**自组织 (Ad hoc) 网络**,或者**移动自组织网络 (MANET, Mobile Ad hoc NETworks)**。现在我们对 Ad hoc 网络进行简略的讨论,更详细的信息请参见 (Perkins, 2001)。

Ad hoc 网络区别于有线网络的原因在于网络拓扑这个概念突然间被彻底抛到了九霄云外。节点可以来来去去,消失一会后突然又出现在一个新的地方。在有线网络中,如果一台路由器有一条通往某个目标的有效路径,那么该路径将会持续有效(除非出现故障)。而在 Ad hoc 网络中,拓扑结构可能每时每刻都在变化,所以路径的可取性和有效性自发地改变着,甚至没有任何警告。毋庸多说,这些情况使得 Ad hoc 网络的路由比固定路由更具挑战性。

针对 Ad hoc 网络目前已经提出了许许多多路由算法。然而,由于 Ad hoc 网络相比移动网络很少被实际使用,因此无法确定哪种协议最有用。作为一个学习案例,我们将考查其中一个最流行的路由算法,即 Ad hoc 按需距离矢量(AODV, Ad hoc Ondemand Distance Vector)路由算法(Perkins 和 Royer, 1999)。它是相对的距离矢量算法,适应在移动环境中工作,即节点的带宽有限和电源寿命相对较短。现在我们来看这个算法如何发现和维护路由的。

路由发现

在 AODV 中,到某个目的地的路由是按需发现的,即只有当某个节点要给目标节点发送数据包时才去找路径。这种方式可以节省大量不必要的工作,比如在路由使用之前拓扑就发生变化,因而之前的所有路由工作都浪费了。在任何时刻,Ad hoc 网络都可以用连接节点的图来描述。如果两个节点可以直接通信,则这两个节点是连接的(也就是说,在图中它们之间有一条弧)。我们采用了一个简单但恰当的模型,即每个节点都可以与位于其覆盖范围内的其他节点通信。真实网络要复杂得多,可能有建筑物、山坡或者其他妨碍通信的障碍物,还可能出现这种情况,节点 A 连接到节点 B,但是节点 B 却无法连接到 A,因为 A 有比 B 更强大的功率。然而,为简化起见,我们假设所有的连接都是对称的。

为了描述 AODV 算法,请考虑图 5-20 中新形成的 Ad hoc 网络。假设节点 A 上的一个进程想要给节点 I 上的一个进程发送数据包。AODV 算法在每个节点上维护了一张距离矢量表,以目标节点作为关键字,每个表项给出了有关该目标节点的信息,包括将数据包发送给哪个邻居才可以到达这个目标。首先, A 检查自己的表,没有发现针对 I 的表项。因此,它现在必须去发现一条通向 I 的路径。这种只有当需要时才找寻路径的特性使得这个算法是“按需”(on demand)进行的。

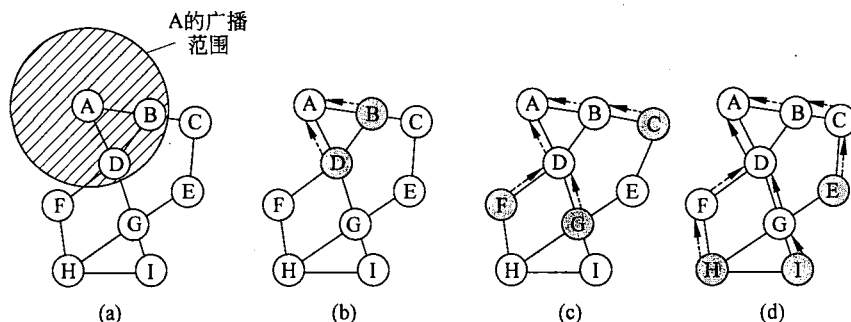


图 5-20

(a) A 的广播范围；(b) B 和 D 接收之后；(c) C、F 和 G 接收之后；
(d) E、H 和 I 接收之后阴影节点是新接收者，虚线表示可能的逆向路由，实线表示发现的路由

为了找到节点 I，A 构造一个 ROUTE REQUEST（路由请求）包，并且使用我们在 5.2.3 节描述的泛洪算法广播它。该包从 A 被传输到 B 和 D，如图 5-20 (a) 所示。每个节点重新广播，该包继续达到节点 F、G 和 C，见图 5-20 (c) 和 H、E 和 I，见图 5-20 (d)。源端设置的一个序号用来淘汰泛洪过程中的重复请求包。例如，D 丢弃来自 B 的传输，如图 5-20 (c)，因为它已经转发过请求数据包了。

最后，请求包到达节点 I，节点 I 构造一个 ROUTE REPLY（路由应答）包。这个包沿着请求包所遵循的路径逆向单播给发送者。这项工作要求每一个中间节点必须记住给它发送请求包的节点。图 5-20 (b) ~ 图 5-20 (d) 显示了存储的逆向路由信息。每一个中间节点在转发应答数据包时还要将跳数递增一，告诉节点到目标节点多远。应答数据包还告诉每一个中间节点，使用哪个邻居能到达目标节点：正是给它们发送应答数据包的节点。中间节点 G 和 D 在处理应答数据包时把听到的最好的路由填入它们的路由表中。当该应答包到达节点 A，一条新路由 ADGI 就被创建了出来。

在一个大型网络中，AODV 算法会产生许多广播包，即使对于比较近的目标也是如此。为了减少开销，可以使用 IP 包的 Time to live 字段来限制广播的范围。该字段由发送者初始化，并且每经过一跳该字段被减 1。如果该字段被减到 0，则把包丢弃，不再广播。因此，路由发现过程可以作如下修改。为了发现一个目标，发送者广播一个 ROUTE REQUEST 包，其中 TTL 字段设置为 1。如果在一段合理的时间内没有应答包返回，则再发送一个 ROUTE REQUEST 包，这次将 TTL 字段设置为 2。以后的每一次尝试分别使用 TTL 的 3、4、5 值等。按照这种方法，搜索过程首先在本地进行，然后逐步扩大到更广的范围。

路由维护

因为节点可以移动或者关闭，所以网络的拓扑结构自然会发生变化。例如，在图 5-20 中，如果 G 关闭了，那么 A 并不会立即意识到它刚刚用过的通向 I 的路径（ADGI）已不再有效了。路由算法必须能够处理这样的情况。每个节点周期地广播一个 HELLO 消息，并且期望它的邻居做出响应。如果没有响应到来，消息的广播者就知道它的邻居已经离开接收范围或者失效，因而不跟自己有连接。类似地，如果它试图给一个邻居发送数据包而没有得到响应，那么它也知道该邻居已经不再可用。

这些信息可被用来清除掉那些不再有效的路由。对于每一个可能的目标节点，每个节

点(比如说 N)记录自己的活跃邻居,即那些在最近的 ΔT 秒内给它发过到达该目标的数据包。当 N 的任何邻居变得不可达时,它就检查自己的路由表看哪些通往目标节点的路由使用了刚刚离开的邻居。对于这些路径中的每一个,通知相应的活跃邻居,因为它们经过 N 的路径不再有效,必须从路由表中删除。在我们的例子中,节点 D 从路由表中删除到 G 和 I 的表项,并通知 A 也删除到 I 的表项。一般情况下,活跃邻居再告诉它们自己的活跃邻居,如此递归下去,直到所有依赖走掉节点的路由从全部的路由表中删除掉。

在这个阶段,无效的路由已被清除出网络,发送者可以使用我们描述过的路由发现机制来发现新的有效路由。然而,这里有点复杂。回想一下,距离矢量协议可能遭受慢收敛问题,或者拓扑发生变化之后的无穷计数问题,这些问题把新的有效路由和旧的无效路由混淆在一起。

为了保证快速收敛,路由包括了一个由目标节点控制的序号。目标序号就像个逻辑时钟。目标节点每次发送一个新 ROUTE REPLY 时就递增该序号。发送者请求发现一条新路由时,就在 ROUTE REQUEST 请求报文中包括它最后使用的那条路由的序号,这个序号要么是刚刚被清除的路由的序号,要么作为初始值被设置为 0。该请求报文将被广播直到发现一条具有更高序号的路由。中间节点存储具有更高序号的路由,或者当前序号下具有更少跳的路由。

按需协议的精髓在于中间节点只存储正在使用中的路由。在广播期间了解的其他路由信息经过短暂延迟后会超时。相比需要定期广播路由更新信息的标准距离向量协议,只发现并存储那些要使用的路由有助于节省带宽和电池寿命。

到目前为止,我们已经考虑了网络中只有单条路由的情况,即从 A 到 I。为了进一步节省资源,路由发现和路由维护可被重叠的路由共享。例如,如果 B 也想给 I 发送数据包,它将执行路由发现。然而,在这种情况下,请求报文首先到达 D,而 D 已经有一条路由可以通向 I。因此节点 D 生成一个路由应答报文,告诉 B 这条路由而无须做任何额外的工作。

还有许多其他的 Ad hoc 路由方案。另一个著名的按需路由方案是动态源路由(DSR, Dynamic Source Routing)(Johnson 等, 2001)。贪婪边界无状态路由(GPSR, Greedy Perimeter Stateless Routing)(Karp 和 Kung, 2000)则探讨了另一种基于地理位置信息的不同的路由策略。如果所有节点知道它们的地理位置,则数据包的转发无须进行路由计算,只需简单地朝着正确的方向,并绕回躲开任何死角即可。究竟哪个路由协议能胜出将取决于 Ad hoc 网络的类型,必须实践证明对这种网络是有用的。

5.3 拥塞控制算法

(一部分)网络中存在太多的数据包导致数据包被延迟延迟和丢失,从而降低了传输性能,这种情况称为拥塞(congestion)。网络层和传输层共同承担着处理拥塞的责任。由于拥塞发生在网络内,正是网络层直接经历着拥塞,而且必须由它最终确定如何处理过载的数据包。然而,控制拥塞的最有效方法是减少传输层注入网络的负载。这就需要网络层和传输层共同努力协同工作。在这一章中,我们将着眼于拥塞控制在网络层方面的处理;在第 6 章,我们将通过覆盖拥塞控制在传输方面的处理来结束拥塞控制主题。

图 5-21 描绘了拥塞的发生。当主机发送到网络的数据包数量在其承载能力范围之内时，送达的数据包数与发送的数据包数成正比例增长。如果发送量增加了两倍，则送达量也增长了两倍。然而，随着负载接近承载能力，偶尔突发的流量填满了路由器内部的缓冲区，因而某些数据包会被丢失。这些丢失的数据包消耗了部分容量，因此，送达的数据包数量低于理想曲线。网络现在开始拥挤了。

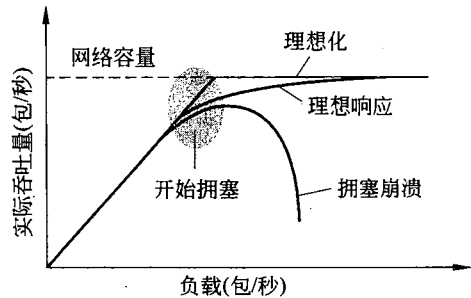


图 5-21 太多的流量导致性能急剧下降

除非网络是精心设计的，否则它极有可能会遭遇拥塞崩溃（congestion collapse），表现为随着注入负载的增加到超出网络的容量，网络性能骤降。这种情况非常有可能发生，因为数据包在网络内部遭遇了足够的延迟，使得它们离开网络后已经不再有用。例如，在早期 Internet 中有许多慢速的 56 kbps 链路，那么通过这样的链路发送数据包，包花在等待积压在前面数据包排空的时间远远超过了允许其在网络中的最大生存期，因此不得被扔掉。这种被延迟很长时间的数据包会被认为已经丢失，因而需要重传。当发送方重传这些数据包时就出现了一个不同的故障模式。在这种情况下，相同数据包的副本将通过网络传送，再次浪费了网络的容量。为了捕捉这些因素，图 5-21 中的 y 轴代表实际吞吐量（goodput），表示网络传递有用（useful）数据包的传送速率。

我们想设计出这样的网络：首先尽可能地避免产生拥挤，其次如果它们确实变得拥挤时不会遭遇拥塞崩溃。不幸的是，拥塞不能完全避免。如果突然间，从三四条线路入境的数据包流都需要转发到相同的输出线路，则会形成一个队列。如果没有足够的内存来容纳所有这些数据包，数据包将被丢弃。为路由器添加更多的内存可以缓解这一点，但（Nagle, 1987）认识到如果路由器拥有无限内存，拥堵情况将更加恶化而不是缓解。这是因为当数据包排到队列前面时，它们早已经超时（重复地）并且它们的副本也已经发送。这样使得事情变得更糟，而不是更好——最终导致拥塞崩溃。

低带宽链路或者处理数据包速度比线路速率还低的路由器也会变得拥挤不堪。在这种情况下，把一些流量导出瓶颈区域指向网络的其他部分可以改善这种拥塞状况。然而，最终网络的所有地区都将出现挤塞。在这种情况下，没有什么办法，只好卸下负载或建立一个更快的网络。

值得指出的是拥塞控制和流量控制之间有很大的差异，它们之间的关系非常微妙。拥塞控制的任务是确保网络能够承载所有到达的流量。这是一个全局性的问题，涉及各方面的行为，包括所有的主机和所有的路由器。与此相反，流量控制只与特定的发送方和特定的接收方之间的点到点流量有关。它的任务是确保一个快速的发送方不会持续地以超过接收方接收能力的速率传输数据。

为了看清楚这两个概念之间的差异，请考虑一个光纤网络，它由 100 Gbps 的光纤链路组成。在这个网络上，一台超级计算机试图给一台个人计算机传送一个大文件，这台个人计算机只有处理 1 Gbps 速率的能力。尽管这里并没有拥塞（网络本身没有任何问题），但是流量控制却十分需要，以便强迫超级计算机经常停下来，给个人计算机以喘息的机会。

作为另一个极端的情形，请考虑这样一个网络：它的线路是 1 Mbps，有 1000 台大型计算机，其中一半的机器试图给另一半机器以 100 kbps 的速率传送文件。这里的问题并不是快速的发送方会淹没慢速的接收方，而是交给网络的总流量超过了网络的处理能力。

拥塞控制和流量控制之所以常常被混淆的原因在于处理上述两个问题的最好方式都是迫使主机慢下来。因此，一台主机接收到的“减速慢行”消息既可能来自不能处理负载的接收方，也可能来自不能处理负载的网络。我们将在第 6 章回到这个话题。

我们首先考查在不同时间尺度可使用的一些方法，作为学习拥塞控制的开始；然后，我们将考查把预防拥塞放在首位的方法，其次学习一旦拥塞发生以后的各种动态处理算法。

5.3.1 拥塞控制的途径

拥塞的出现意味着负载（暂时）大于资源（在网络的一部分）可以处理的能力。很自然人们能想到两个解决方案：增加资源或减少负载。如图 5-22 所示，这些解决方案通常应用在不同的时间尺度上，要么预先避免拥塞，要么一旦发生拥塞随之做出反应。

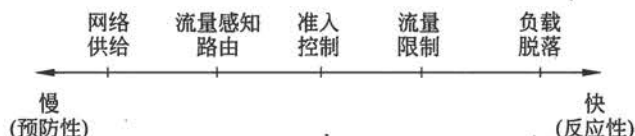


图 5-22 拥塞控制方法的时间尺度

避免拥塞的最基本方法是建立一个与流量匹配良好的网络。如果存在一条低带宽的链路，大多数流量所走的路径都要经过这条链路，那么发生拥塞的可能性非常大。有时当出现严重拥塞时，可以动态增加网络资源，例如，把通常只用来备份的路由器或线路（使系统容错）打开，或者在公开市场上购买带宽。更多的时候，经常大量使用的链路和路由器都尽早实行升级。这就是所谓的供给（provisioning），在长期流量趋势推动下大约需要几个月的时间。

为了充分利用现有的网络容量，根据每天的流量模式度身定制路由，因为不同时区的网络用户每天醒来和睡觉的时间是不同的。例如，通过改变最短路径的权重可更改数据包路由，以便使流量远离频繁使用的路径。一些本地广播电台有直升机围绕着城市上空飞来飞去，及时报告城市道路拥堵情况，帮助它们的移动听众路由它们的数据包（汽车）绕开热点地区。这称为流量感知的路由（traffic-aware routing）。把流量拆分到多个路径也是有用的。

然而，有的时候不可能增加容量。那么对抗拥塞的唯一的办法就是降低负载。在一个虚电路网络中，如果新的连接将导致网络变得拥挤不堪，那么就应该拒绝这种新连接的建立。这种控制称为准入控制（admission control）。

以更细一点的粒度，当拥塞已迫在眉睫时，网络可以给造成问题的数据包源端传递反馈信息，要求这些源端抑制它们的流量，或者减缓流量本身。

这种方法存在两个困难：第一，如何确定网络开始拥塞了；第二，如何通知源端减缓速度。为了解决第一个问题，路由器可监测平均负荷、排队延迟或丢包情况。在所有情况下，数值上升表明拥挤情况越来越严重。

为了解决第二个问题，路由器必须参与到源端的反馈循环中。为了正常工作，必须仔细调整时间尺度。如果每次连续到达两个数据包，路由器就叫喊 STOP；而每当路由器空闲 20 微秒时，它就喊 GO，那么系统将会剧烈地摇摆不定，无法收敛。另一方面，如果它等待 30 分钟才能确定说什么，则拥塞控制机制的反应过于迟缓，一点用途也没有。及时提供反馈并不是件小事。需要额外关注的是，当网络早已被堵塞时路由器或许要发送更多的消息。

最后，当一切努力都失败，网络不得不丢弃它无法传递的数据包。这种方法的通用名称是负载脱落 (load shedding)。一个选择丢弃哪些数据包的良好策略可以防止拥塞崩溃。

5.3.2 流量感知路由

我们考查的第一种方法是流量感知路由。我们在 5.2 节看过的路由方案采用固定链路权重。这些方案能适应拓扑结构的变化，但不能适应负载的变化。在计算路由时考虑链路负载的目的是把热点地区的流量转移出去，而热点地区是指网络中体验到拥塞的第一位置。

最直接的方式是把链路权重设置成一个（固定）链路带宽、传输延迟、（可变）测量负载或平均排队延迟的函数。在所有其他条件都相同的情况下，最小权重的路径更青睐那些轻负载的路径。

早期 Internet 使用的流量感知路由就是按照这个模型设计的 (Khanna 和 Zinky, 1989)。然而，这种路由存在一个危险。考虑图 5-23 所示的网络，这里网络被分为东部和西部，这两部分通过链路 CF 和 EI 相连。假设东西之间的大部分流量使用链路 CF，因此，这个链路负荷超重因而延迟增大。如果把排队延迟加入到计算最短路径的权重中，那么链路 EI 将变得更具吸引力。当新的路由表被安装好之后，大部分东西方的流量现在改走链路 EI，由此增加了此链路的负载。因此，在下一次路由更新时，CF 将成为最短路径。结果，路由表可能会剧烈地摇摆不定，从而导致不稳定的路由和许多潜在的问题。

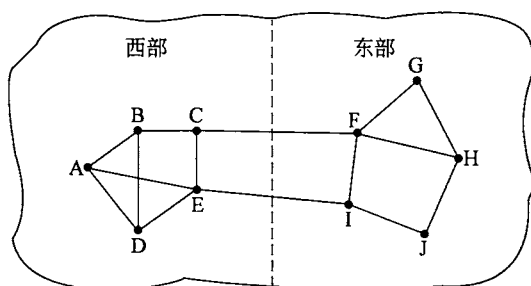


图 5-23 一个网络的東西两个部分由两条链路连接

如果忽略负载，只考虑带宽和传输延迟，这个问题就不会发生。尝试在路由权重中包括负载但将其限定在一个狭窄的范围可减缓路由振荡。两种技术有助于获得成功的解决方案。首先是多路径路由，即从源到目的地可以存在多条路径。在我们的例子中，这意味着可以把整个流量分散到东部和西部的两条链路上。第二个技术是把流量慢慢迁移，足够慢到路由算法得到收敛，比如 (Gallagher, 1977) 方法。

由于存在这些困难，Internet 路由协议通常不依赖于负载来调整自己的路由。相反，在

路由协议外部通过慢慢改变它的输入来调整路由。这种方法就是所谓的流量工程（traffic engineering）。

5.3.3 准入控制

一种广泛应用于虚电路网络，防止出现拥塞的技术是准入控制（admission control）。其基本思想非常简单：除非网络可以携带额外的流量而不会变得拥塞，否则不再建立新的虚电路。因此，任何建立新的虚电路的尝试或许会失败。这种方法比起那种让更多的人进入一个已经繁忙的网络更好，因为更多的人只会使情况变得更糟糕。类似地，在电话系统中，当一台交换机实际超载时，它也会采用准入控制的方法，不再送出拨号音。

这种方法的诀窍是当一条新的虚电路将导致拥塞时如何工作。这项任务在电话网络中比较简单，因为电话呼叫所需的带宽固定（64 kbps 的无压缩音频）。然而，计算机网络中的虚拟电路有各种形状和大小。因此，如果我们想要采用准入控制，必须归纳出虚电路的一些流量特性。

流量往往用其速度和形状来描述。如何以一种简单而又有意义的方式来描述流量是困难的，因为流量呈现突发性——平均速率只讲述了故事的一半。例如，浏览网页时的流量变化很大，比具有固定长期吞吐量的流式电影更难以处理，因为突发性的网页流量更容易堵塞住网络中的路由器。捕获这个效果通常采用的描述符是漏桶（leaky bucket）或令牌桶（token bucket）。一个漏桶有两个参数约束了平均速率和瞬时突发流量大小。由于漏桶被广泛应用于服务质量，我们将在 5.4 节仔细描述它。

有了流量说明，网络就能决定是否接受新的虚电路。一种可能性是网络为它的每条虚电路保留其沿途的足够容量，这样就不会出现拥塞。在这种情况下，流量说明是一个网络向用户提供保证的服务约定。我们已经预防拥塞，但有点过早地转向了有关服务质量的话题；我们将在下一节返回到这里。

即使没有做出保证，网络也可以利用流量说明来进行准入控制。然后，我们的任务是估计出多少条电路能被网络的承载能力所容纳，而不会拥塞。假设虚拟电路可能爆发的流量速率高达 10 Mbps，如果所有流量都要通过同一条 100 Mbps 的物理链路，那么应该准许多少条电路？显然，10 条电路是可以准许的，并且不会有拥塞的危险；但这在正常情况下会浪费带宽，因为可能很少发生所有 10 个用户在同一时间以全速率传送数据的情形。在实际网络中，对过去行为的测量，即捕获数据传送的统计特征，可用来估计准入的虚电路数量，从而以可接受的风险换取更好的性能。

准入控制还可以和流量感知路由相结合，在虚电路建立过程中，考虑绕开流量热点区域的路由。例如，考虑图 5-24 的网络，这里显示的两台路由器已经被堵塞。

假设一台连接到路由器 A 的主机想要与另一台连接到路由器 B 的主机建立一个连接。通常情况下，这个连接将会经过其中一台拥塞的路由器。为了避免这种情形，我们可以重画网络，如图 5-24（b）所示，去掉拥塞的路由器和它们所有的线路。图中的虚线显示了一条可能的虚电路路径，它避开了拥塞的路由器。（Shaikh 等，1999）给出了这类负载敏感的路由方案设计。